Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта

ОТЧЁТ

по лабораторной работе № 1

«Генерация бинарного кода Грея. Мультимножества и операции над ними»

по дисциплине «Дискретная математика»

Выполнил
студент гр. 5130201/50302 _____ Р. Ю. Дужнов

Проверил
преподаватель _____ А. В. Востров

« ____ » _____ 2026 г.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Математическое описание 4
1.1 Множества и мультимножества 4
1.2 Операции над мультимножествами 5
1.3 Бинарный код Грея 7
1.4 Алгоритм автоматического заполнения 9
2 Особенности реализации 10
2.1 Средства разработки и структура программы 10
2.2 Допущения и входные данные 11
2.3 Структуры данных 12
2.4 Функции и методы 13
2.5 Ограничения программы 18
2.6 Обработка ошибок и защита ввода 20
3 Результаты работы 21
3.1 Корректная работа при $n = 3$ 21
3.2 Некорректный ввод команд и параметров 24
3.3 Ошибки при ручном заполнении 25
3.4 Граничные значения мощности и кратности 27
3.5 Пустой универсум, пересоздание и конец ввода 29
3.6 Наибольшая разрядность 30
ЗАКЛЮЧЕНИЕ 34
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 36
ПРИЛОЖЕНИЕ А Текст программы 37

ВВЕДЕНИЕ

В работе требуется разработать программу для работы с мультимножествами, элементами которых служат слова бинарного кода Грея. Программа должна:

– генерировать бинарный код Грея разрядности n, которую задаёт пользователь, и заполнять его словами универсум мультимножеств;
– формировать на основе универсума два мультимножества A и B, мощности которых задаёт пользователь, а способ заполнения каждого из них — вручную или автоматически — пользователь выбирает сам;
– вычислять и выводить результаты операций над A и B: объединения, пересечения, разностей $A \setminus B$ и $B \setminus A$, симметрической разности, дополнений $\overline{A}$ и $\overline{B}$, арифметических суммы, разности, произведения и деления;
– защищать работу от некорректного ввода и предоставлять пользователю меню.

1 Математическое описание

1.1 Множества и мультимножества

Множество — это любая определённая совокупность объектов. Объекты, из которых составлено множество, называются его элементами. Элементы множества различны и отличимы друг от друга [1]. Запись $x \in X$ означает, что x — элемент множества X.

Универсум U — заранее зафиксированное множество всех элементов, допустимых в конкретной задаче; каждое рассматриваемое в задаче множество является подмножеством U [1].

Пусть $X = \{x_1, \dots, x_N\}$ — конечное множество, $a_1, \dots, a_N$ — неотрицательные целые числа. Мультимножество A над X — совокупность элементов X, в которой x_i содержится ровно a_i раз [1]:

$$A = \langle a_1(x_1), a_2(x_2), \dots, a_N(x_N) \rangle. \tag{1}$$

Множество X — носитель A, число a_i — кратность элемента x_i [1, 2].

Мультимножество A над носителем X задаётся функцией кратности, которая каждому элементу носителя ставит в соответствие число его вхождений в A:

$$k_A : X \to \{0, 1, 2, \dots\}, \qquad k_A(x_i) = a_i, \quad i = 1, \dots, N. \tag{2}$$

Элемент x принадлежит A тогда и только тогда, когда $k_A(x) > 0$.

Мощность мультимножества — сумма кратностей элементов носителя [1]:

$$|A| = \sum_{i=1}^{N} k_A(x_i). \tag{3}$$

Мультимножество A — подмультимножество мультимножества B над тем же носителем, если

$$A \subseteq B \iff k_A(x) \leqslant k_B(x) \ \text{ для всех } x \in X. \tag{4}$$

Из (3) и (4): если $A \subseteq B$, то $|A| \leqslant |B|$.

Универсум мультимножеств — мультимножество U над носителем X, для которого $A \subseteq U$ для каждого рассматриваемого в задаче мультимножества A. По (4) это означает, что $0 \leqslant k_A(x) \leqslant k_U(x)$ для всех $x \in X$, то есть

$k_U(x)$ — наибольшая кратность, с которой элемент x может входить в мультимножества задачи. Отсюда $0 \leqslant |A| \leqslant |U|$.

В программе носитель — множество слов бинарного кода Грея разрядности n (подраздел 1.3); при $n \geqslant 1$ число слов $N = 2^n$. Все слова входят в универсум с одной и той же кратностью k, которую вводит пользователь; если вместо k введена пустая строка, программа выбирает k случайно от 1 до 100:

$$k_U(x) = k \ \text{ для всех } x \in X, \qquad |U| = k \cdot 2^n, \qquad n \geqslant 1. \tag{5}$$

Мультимножества A и B строятся как подмультимножества U, поэтому $k_A(x) \leqslant k$, $k_B(x) \leqslant k$ и $0 \leqslant |A|, |B| \leqslant |U|$. При $n = 0$ алгоритм из [1] выдаёт один код — код пустого подмножества ($2^0 = 1$); в программе принято, что код Грея при $n = 0$ пуст: $N = 0$, $|U| = 0$.

1.2 Операции над мультимножествами

Операции над множествами определяются так [1]:

$$A \cup B = \{x \mid x \in A \vee x \in B\}, \qquad A \cap B = \{x \mid x \in A \ \&\ x \in B\},$$
$$A \setminus B = \{x \mid x \in A \ \&\ x \notin B\}, \qquad A \triangle B = (A \cup B) \setminus (A \cap B), \qquad \overline{A} = U \setminus A.$$

Для мультимножеств A и B над общим носителем X, где $A \subseteq U$ и $B \subseteq U$, кратность элемента x в результате каждой операции вычисляется в программе по кратностям этого же элемента в A, B и U. Объединение, пересечение и дополнение:

$$k_{A \cup B}(x) = \max(k_A(x),\ k_B(x)), \tag{6}$$
$$k_{A \cap B}(x) = \min(k_A(x),\ k_B(x)), \tag{7}$$
$$k_{\overline{A}}(x) = k_U(x) - k_A(x). \tag{8}$$

Кратность (8) неотрицательна, так как $k_A(x) \leqslant k_U(x)$.

Для множеств из определений разности и дополнения следует равенство

$$A \setminus B = A \cap \overline{B}, \tag{9}$$

так как обе части состоят из элементов U, которые принадлежат A и не принадлежат B. В программе разность мультимножеств вычисляется именно в

форме (9). Подставляя в (9) формулы пересечения (7) и дополнения (8), получаем

$$k_{A\setminus B}(x) = \min(k_A(x),\ k_{\overline{B}}(x)) = \min(k_A(x),\ k_U(x) - k_B(x)), \tag{10}$$

и аналогично $k_{B\setminus A}(x) = \min(k_B(x),\ k_U(x) - k_A(x))$.

Для множеств из определения симметрической разности следует равенство

$$A \triangle B = (A \setminus B) \cup (B \setminus A), \tag{11}$$

так как обе части состоят из элементов, которые принадлежат ровно одному из множеств A и B. В программе симметрическая разность мультимножеств вычисляется именно в форме (11). Подставляя в (11) формулы объединения (6) и разности (10), получаем

$$k_{A\triangle B}(x) = \max\Big(\min(k_A(x),\ k_U(x)-k_B(x)),\ \min(k_B(x),\ k_U(x)-k_A(x))\Big). \tag{12}$$

Кратность (12) в общем случае не равна $|k_A(x)-k_B(x)|$: при $k_U(x) = 3, k_A(x) = 1, k_B(x) = 2$ она равна $\max(\min(1,1),\min(2,2)) = 2$.

Арифметические операции выполняются над кратностями:

$$k_{A+B}(x) = \min(k_A(x) + k_B(x),\ k_U(x)), \tag{13}$$

$$k_{A-B}(x) = \max(k_A(x) - k_B(x),\ 0), \tag{14}$$

$$k_{A\cdot B}(x) = \min(k_A(x) \cdot k_B(x),\ k_U(x)), \tag{15}$$

$$k_{A/B}(x) = \begin{cases} k_A(x) \operatorname{div} k_B(x), & k_B(x) > 0, \\ 0, & k_B(x) = 0, \end{cases} \tag{16}$$

где div — целочисленное деление. Сумма и произведение могут превысить $k_U(x)$, поэтому ограничены этой величиной сверху; разность ограничена снизу нулём, так как кратность не может быть отрицательной. Операции $B - A$ и B/A получаются перестановкой A и B. Разность (10) и арифметическая разность (14) — разные операции: при $k_U(x) = 3$, $k_A(x) = 2$, $k_B(x) = 1$ имеем $k_{A\setminus B}(x) = 2$ и $k_{A-B}(x) = 1$.

Формулы всех операций в том виде, в каком они вычисляются в программе, собраны в таблице 1.

Таблица 1 — Операции над мультимножествами

Операция	**Обозначение**	**Кратность элемента x в результате**
Объединение	$A \cup B$	$\max(k_A(x), k_B(x))$
Пересечение	$A \cap B$	$\min(k_A(x), k_B(x))$
Разность	$A \setminus B$	$\min(k_A(x),\ k_U(x) - k_B(x))$
Разность	$B \setminus A$	$\min(k_B(x),\ k_U(x) - k_A(x))$
Симметрическая разность	$A \triangle B$	$\max(k_{A\setminus B}(x),\ k_{B\setminus A}(x))$
Дополнение	$\overline{A}$	$k_U(x) - k_A(x)$
Дополнение	$\overline{B}$	$k_U(x) - k_B(x)$
Арифметическая сумма	$A + B$	$\min(k_A(x) + k_B(x),\ k_U(x))$
Арифметическая разность	$A - B$	$\max(k_A(x) - k_B(x),\ 0)$
Арифметическая разность	$B - A$	$\max(k_B(x) - k_A(x),\ 0)$
Арифметическое произведение	$A \cdot B$	$\min(k_A(x) \cdot k_B(x),\ k_U(x))$
Арифметическое деление	A/B	$k_A(x) \operatorname{div} k_B(x)$; 0 при $k_B(x) = 0$
Арифметическое деление	B/A	$k_B(x) \operatorname{div} k_A(x)$; 0 при $k_A(x) = 0$

Для множеств пересечение, объединение и разность подмножеств универсума являются его подмножествами [1]. Для мультимножеств выполняется аналогичное утверждение: если $A \subseteq U$ и $B \subseteq U$, то результаты всех операций (6)–(16) являются подмультимножествами U: максимум и минимум чисел, не превосходящих $k_U(x)$, не превосходят $k_U(x)$; $0 \leqslant k_U(x) - k_A(x) \leqslant k_U(x)$; обе разности не больше уменьшаемого: $\min(k_A(x), \ldots) \leqslant k_A(x)$ и $\max(k_A(x) - k_B(x), 0) \leqslant k_A(x)$; сумма и произведение ограничены величиной $k_U(x)$ по определению; при $k_B(x) \geqslant 1$ частное $k_A(x) \operatorname{div} k_B(x) \leqslant k_A(x)$, а при $k_B(x) = 0$ оно равно нулю.

1.3 Бинарный код Грея

Бинарным кодом Грея разрядности n называется последовательность всех 2^n двоичных кодов длины n, в которой любые два соседних кода различаются ровно в одном разряде [1].

Код Грея строится алгоритмом из [1] (алгоритмы 1.2 и 1.3). Начальный код состоит из нулей. На шаге $i = 1, 2, \ldots, 2^n - 1$ меняется разряд с номером $Q(i)$, где $Q(i)$ — количество нулей на конце двоичной записи числа i, увеличенное на 1; в примере выполнения алгоритма в [1] разряд с номером 1 — крайний правый. Корректность алгоритма доказана в [1] индукцией по n.

В программе разряды нумеруются справа, начиная с 0, поэтому номер

разряда, который меняется на шаге i, равен

$$p(i) = Q(i) - 1 = \text{количество нулей на конце двоичной записи числа } i. \quad (17)$$

При $n \geqslant 1$, если обозначить i-й код последовательности через g_i,

$$g_0 = 0, \qquad g_i = g_{i-1} \oplus 2^{p(i)}, \quad i = 1, 2, \ldots, 2^n - 1, \qquad (18)$$

где $\oplus$ — побитовое исключающее ИЛИ. Код строит функция `generate_gray_code`, номер разряда $p(i)$ вычисляет функция `changed_bit`. Алгоритм в программе выполняется так:

1) если $n = 0$, возвращается пустая последовательность, и алгоритм завершается;
2) если $n \geqslant 1$, в последовательность записывается код $g_0 = 0$;
3) для $i = 1, 2, \ldots, 2^n - 1$: число i делится на 2, пока результат чётный, число делений равно $p(i)$; в текущем коде инвертируется разряд $p(i)$, и полученный код g_i дописывается в последовательность;
4) последовательность возвращается.

Чтобы найти $p(i)$, число i проверяется на чётность $p(i)+1$ раз. Проверка с номером $t + 1$, где $t = 0, 1, \ldots, n - 1$, выполняется только для тех i, которые делятся на 2^t; среди чисел от 1 до $2^n - 1$ таких чисел $2^{n-t} - 1$. Поэтому общее число проверок

$$\sum_{i=1}^{2^n-1} (p(i) + 1) = \sum_{t=0}^{n-1} (2^{n-t} - 1) = 2^{n+1} - n - 2 < 2 \cdot 2^n. \qquad (19)$$

Кроме проверок, на каждом из $2^n - 1$ шагов выполняются одно инвертирование разряда и одна запись кода. Трудоёмкость алгоритма — $O(2^n)$, память — 2^n чисел под получаемую последовательность (носитель в программе хранит ещё 2^n чисел обратной таблицы, подраздел 2.3). Пошаговое выполнение алгоритма при $n = 3$ приведено в таблице 2; оно совпадает с примером выполнения алгоритма 1.2 в [1] и с выводом программы на рисунке 2.

Таблица 2 — Пошаговое построение кода Грея при $n = 3$

i	i в двоичной записи	$Q(i)$	$p(i)$	g_{i-1}	g_i
0	000	—	—	—	000
1	001	1	0	000	001
2	010	2	1	001	011
3	011	1	0	011	010
4	100	3	2	010	110
5	101	1	0	110	111
6	110	2	1	111	101
7	111	1	0	101	100

1.4 Алгоритм автоматического заполнения

Автоматическое заполнение строит случайное подмультимножество $R \subseteq U$ заданной мощности m, где $0 \leqslant m \leqslant |U|$. В программе его выполняет функция `random_submultiset`. Алгоритм:

1) создаётся мультимножество R над носителем U с нулевыми кратностями и список F элементов, кратность которых в R ещё меньше, чем в U; в начале в F входят все x, у которых $k_U(x) > 0$;
2) m раз выполняется: из списка F равновероятно выбирается элемент x, его кратность $k_R(x)$ увеличивается на 1; если $k_R(x)$ стала равна $k_U(x)$, элемент x удаляется из F: на его место в списке ставится последний элемент, и список укорачивается на 1;
3) возвращается R.

Шаги нумеруются $s = 0, 1, \ldots, m - 1$. Перед шагом s в R распределено s единиц, поэтому свободных мест остаётся $|U| - s > 0$, так как $s < m \leqslant |U|$. Следовательно, список F не пуст ни на одном шаге, кратность ни одного элемента не превышает $k_U(x)$, $R \subseteq U$ и $|R| = m$.

До начала шагов выполняется $O(2^n)$ операций: вычисление $|U|$ для проверки условия $m \leqslant |U|$ (сумма 2^n кратностей), создание R с 2^n нулевыми кратностями и построение списка F. Каждый из m шагов состоит из выбора случайной позиции в F, увеличения одной кратности и, возможно, удаления элемента переносом последнего; эти действия выполняются за $O(1)$, причём выбор случайного числа — в среднем за $O(1)$. Трудоёмкость алгоритма — $O(2^n + m)$, дополнительная память — $O(2^n)$ под R и F.

2 Особенности реализации

2.1 Средства разработки и структура программы

Программа написана на языке C++ стандарта C++14. Для сборки в проекте есть файл `CMakeLists.txt`: он задаёт стандарт C++14 и две цели — саму программу `lab1` и программу замера времени `bench`. Для отчёта программа собиралась компилятором g++ (MinGW.org GCC 6.3.0, 32-разрядная сборка) напрямую:

```
g++ -std=c++14 -O2 -Wall -Wextra -Isrc src/main.cpp
    src/ui/*.cpp src/core/*.cpp -o lab1.exe
```

Предупреждений при компиляции нет. Программа запускается без параметров либо с параметром `--seed N`: тогда генератор случайных чисел получает зерно N, и запуск с тем же вводом повторяется полностью. Без параметра зерно получает функция `make_seed`. В обоих случаях зерно выводится первой строкой.

Исходный код разделён на ядро (`src/core`), где реализованы код Грея и мультимножества, и интерфейс (`src/ui`), где реализованы ввод и меню. Ядро не выводит ничего на экран, а интерфейс не вычисляет операции сам. Назначение файлов приведено в таблице 3.

Таблица 3 — Файлы проекта

Файл	**Назначение**	**Листинг**
`CMakeLists.txt`	сборка программы `lab1` и программы замера `bench`	3
`src/main.cpp`	функция `main`: кодировка консоли, зерно, запуск меню, перехват исключений	4
`src/ui/Input.hpp`	объявления функций ввода, класс `Input` `ClosedError`	5
`src/ui/Input.cpp`	чтение строк и чисел с проверкой	6
`src/ui/Menu.hpp`	объявление класса `Menu`	7
`src/ui/Menu.cpp`	меню, заполнение мультимножеств, вывод таблицы, операций и матрицы	8
`src/core/GrayCode.hpp`	константы `MAX_DEPTH`, `MAX_TYPE_DEPTH`, классы исключений, класс `CodeSet`	9
`src/core/GrayCode.cpp`	генерация кода Грея, перевод кода в строку, методы `CodeSet`	10
`src/core/Multiset.hpp`	класс `Multiset`, классы исключений	11

Файл	Назначение	Листинг
`src/core/Multiset.cpp`	операции над мультимножествами, автоматическое заполнение, печать	12
`src/core/Random.hpp`	объявления функций генератора случайных чисел	13
`src/core/Random.cpp`	генератор `std::mt19937` и функции работы с ним	14
`src/core/Universe.hpp`	константа `MAX_MULTIPLICITY`, класс `InvalidMultiplicityError`	15
`src/core/Universe.cpp`	построение равномерного универсума	16
`tools/bench.cpp`	замер времени самых долгих действий при разных n	17

2.2 Допущения и входные данные

При реализации приняты следующие допущения. Универсум равномерный: все слова кода Грея входят в него с одной и той же кратностью k, которую задаёт пользователь или выбирает программа. Разрядность ограничена условием $0 \leqslant n \leqslant 19$, кратность — условием $1 \leqslant k \leqslant 100$; выбор этих пределов обоснован в подразделе 2.5. При $n = 0$ код Грея по соглашению из подраздела 1.1 считается пустым, поэтому универсум пуст, кратность не запрашивается, а мощности A и B могут быть только нулевыми. Мультимножества A и B всегда строятся как подмультимножества универсума, поэтому их мощности лежат в пределах от 0 до $|U|$. Пересоздание универсума сбрасывает уже заданные A и B, так как они построены над прежним носителем. Если заполнение отменено, прежнее мультимножество сохраняется.

Входные данные программы, их допустимые значения и реакция программы на некорректный и пустой ввод приведены в таблице 4.

Таблица 4 — Входные данные программы

Величина	Допустимые значения	Реакция на некорректный и пустой ввод
Команда меню	одна буква из `u a b t o m h q`, регистр не важен	сообщение «не команда» или «пустая строка» и повторный запрос; если команда недоступна, выводится, чего не хватает
Разрядность n	целое от 0 до 19	повторный запрос с причиной: пустая строка, не число, число вне отрезка
Кратность k	целое от 1 до 100; при $n = 0$ не запрашивается	пустая строка — случайное k от 1 до 100; иначе повторный запрос с причиной
Мощности $\|A\|$, $\|B\|$	целое от 0 до $\|U\|$	пустая строка — случайная мощность от 0 до $\|U\|$; иначе повторный запрос

Величина	**Допустимые значения**	**Реакция на некорректный и пустой ввод**
Способ заполнения	`r` — случайно, `m` — вручную	пустая строка — отмена, мультимножество не меняется; иначе повторный запрос
Строка ручного ввода	пары `код:кратность` через пробел, код из n символов 0 и 1, кратность от 1	строка отклоняется целиком с указанием причины; пустая строка — отмена

2.3 Структуры данных

Носитель мультимножеств хранится в классе `CodeSet`. Его поля: `depth` — разрядность n; `codes` — вектор `std::vector<size_t>` из 2^n слов кода Грея в порядке генерации; `positions` — вектор той же длины, где `positions[c]` равно номеру слова c в `codes`. Так как в коде Грея встречаются все 2^n слов, номер есть у любого слова, и введённое вручную слово переводится в номер элемента за $O(n)$ операций, без поиска по последовательности. Слово хранится как число, а в строку из нулей и единиц переводится только при выводе.

Мультимножество хранится в классе `Multiset` как функция кратности, то есть массивом кратностей элементов носителя; такое представление мультимножества описано в [1]. Поле `carrier` типа `std::shared_ptr<const CodeSet>` указывает на общий носитель, а поле `counts` типа `std::vector<size_t>` содержит кратности: `counts[i]` — кратность i-го слова кода Грея. Универсум U, мультимножества A и B и все результаты операций — объекты этого класса над одним носителем. Операции над мультимножествами с разными носителями запрещены: это проверяется сравнением указателей.

Состояние программы хранит класс `Menu`. В нём есть указатель на носитель `carrier`, мультимножества `universe`, `a`, `b` и флаги `universe_ready`, `a_ready`, `b_ready`, которые показывают, что соответствующее мультимножество уже задано. Список операций для вывода хранится в массиве `OPERATIONS` из 13 структур `OperationInfo` с полями `title` (название), `notation` (обозначение) и `formula` (формула для справки).

Ошибки передаются исключениями. Все классы исключений наследуются от `std::runtime_error`. От класса `GrayCodeError` наследуются `InvalidDepthError` и `CodeIndexError`. От класса `MultisetError` наследу-

ются MultisetIndexError, CarrierMismatchError, NotSubmultisetError, CardinalityOverflowError и InvalidMultiplicityError. Отдельно стоит InputClosedError, который означает конец ввода.

2.4 Функции и методы

Функции и методы ядра описаны в таблицах 5–7, функции ввода — в таблице 8, методы класса Menu — в таблице 9, служебные функции вывода и функция main — в таблице 10. Для каждой функции указаны назначение, входные данные, возвращаемое значение и исключения, которые она может выбросить.

Таблица 5 — Функции и методы файла GrayCode.cpp

Имя	Назначение	Вход → выход	Исключения
changed_bit (static)	определяет номер разряда $p(i)$, который меняется на i-м шаге генерации: делит i на 2, пока оно чётное, и считает деления	size_t i ($i \geqslant 1$) → size_t — количество нулей в конце двоичной записи i	нет
generate_gray_code	строит код Грея: начинает со слова 0 и на каждом шаге инвертирует разряд changed_bit(i)	size_t depth → std::vector<size_t> из 2^{depth} слов в порядке кода; при depth = 0 вектор пуст	InvalidDepth Error при depth > MAX_TYPE_ DEPTH
code_to_string	переводит слово, хранящееся числом, в строку из символов 0 и 1, старший разряд слева	size_t code, size_t depth → std::string длины depth	нет
CodeSet::CodeSet	конструктор носителя: вызывает generate_gray_code и заполняет обратную таблицу positions	size_t depth	InvalidDepth Error
CodeSet::get_depth	возвращает разрядность n	→ size_t	нет
CodeSet::size	возвращает число слов носителя	→ size_t: 2^n, при $n = 0$ — 0	нет
CodeSet::get_code	возвращает i-е слово кода Грея в виде строки	size_t i → std::string	CodeIndex Error при $i \geqslant$ size()
CodeSet::find	по строке из 0 и 1 находит номер слова в коде Грея через таблицу positions за $O(n)$	const std::string& code, size_t& index → bool: true и номер в index; false, если длина строки не n, в ней есть другие символы или носитель пуст	нет

Таблица 6 — Методы класса Multiset и функции файла Multiset.cpp

Имя	Назначение	Вход → выход	Исключения
Multiset (конструктор)	создаёт мультимножество над носителем, все кратности равны 0	std::shared_ptr<const CodeSet> carrier	нет
check_same_carrier (private)	проверяет, что у двух мультимножеств один и тот же носитель; сравниваются указатели	const Multiset& other	Carrier MismatchError

Имя	Назначение	Вход → выход	Исключения
`check_within` (private)	проверяет, что мультимножество является подмультимножеством универсума (4)	`const Multiset& universe`	`Not Submultiset Error`, `Carrier MismatchError`
`get_carrier`	возвращает ссылку на носитель; нужна при печати слов	→ `const CodeSet&`	нет
`size`	возвращает число элементов носителя	→ `size_t`	нет
`get_count`	возвращает кратность i-го слова	`size_t i` → `size_t`	`MultisetIndex Error` при $i \geqslant$ `size()`
`set_count`	записывает кратность i-го слова; проверки по универсуму не делает	`size_t i`, `size_t count`	`MultisetIndex Error`
`cardinality`	вычисляет мощность (3) — сумму всех кратностей	→ `size_t`	нет
`is_submultiset_of`	поэлементно проверяет условие (4)	`const Multiset& other` → `bool`	`Carrier MismatchError`
`unite`	объединение: в каждой позиции берётся $\max(k_A(x), k_B(x))$	`const Multiset& b` → `Multiset`	`Carrier MismatchError`
`intersect`	пересечение: в каждой позиции берётся $\min(k_A(x), k_B(x))$	`const Multiset& b` → `Multiset`	`Carrier MismatchError`
`complement`	дополнение до универсума: $k_U(x) - k_A(x)$; перед вычислением проверяет $A \subseteq U$	`const Multiset& universe` → `Multiset`	`Not Submultiset Error`, `Carrier MismatchError`
`difference`	разность $A \setminus B$, вычисляется как `intersect( b.complement(universe))`	`const Multiset& b`, `const Multiset& universe` → `Multiset`	как у `complement`
`symmetric_ difference`	симметрическая разность: объединение разностей $A \setminus B$ и $B \setminus A$	`const Multiset& b`, `const Multiset& universe` → `Multiset`	как у `complement`
`arithmetic_sum`	сумма $\min(k_A(x) + k_B(x), k_U(x))$; перед вычислением проверяет $A \subseteq U$ и $B \subseteq U$	`const Multiset& b`, `const Multiset& universe` → `Multiset`	`Not Submultiset Error`, `Carrier MismatchError`
`arithmetic_ difference`	разность $\max(k_A(x) - k_B(x), 0)$; кратности сравниваются до вычитания, поэтому беззнаковый тип не переполняется	`const Multiset& b` → `Multiset`	`Carrier MismatchError`
`arithmetic_product`	произведение $\min(k_A(x) \cdot k_B(x), k_U(x))$; проверки как у суммы	`const Multiset& b`, `const Multiset& universe` → `Multiset`	`Not Submultiset Error`, `Carrier MismatchError`
`arithmetic_ division`	деление $k_A(x) \operatorname{div} k_B(x)$, при $k_B(x) = 0$ результат 0	`const Multiset& b` → `Multiset`	`Carrier MismatchError`
`random_submultiset` (friend)	автоматическое заполнение по алгоритму подраздела 1.4	`const Multiset& bound`, `size_t cardinality` → `Multiset` мощности cardinality	`Cardinality OverflowError` при cardinality > |bound|
`print_multiset`	печатает запись вида `{000×2, 011×1}`, пропуская нулевые кратности; после limit элементов печатает «`... ещё N эл.`»	`std::ostream& out`, `const Multiset& multiset`, `size_t limit`	нет

Имя	Назначение	Вход → выход	Исключения
operator<<	печатает мультимножество полностью, вызывая print_multiset с limit, равным size()	std::ostream& out, const Multiset& multiset → std::ostream&	нет

Таблица 7 — Функции файлов Universe.cpp и Random.cpp

Имя	Назначение	Вход → выход	Исключения
uniform_universe	создаёт универсум (5): каждое слово носителя получает кратность multiplicity	std::shared_ptr<const CodeSet> carrier, size_t multiplicity → Multiset	Invalid Multiplicity Error, если multiplicity вне [1, 100]
generator (static)	хранит единственный объект генератора std::mt19937 в статической переменной	→ std::mt19937&	нет
make_seed	получает зерно: значение std::random_device, к которому примешано текущее время (в MinGW random_device при каждом запуске выдаёт одно и то же)	→ unsigned	нет
seed_random	задаёт начальное состояние генератора; одно и то же зерно даёт одну и ту же последовательность чисел	unsigned seed	нет
random_below	возвращает равновероятное целое число от 0 до bound − 1 через std::uniform_int_distribution	size_t bound (> 0) → size_t	нет

Таблица 8 — Функции файла Input.cpp

Имя	Назначение	Вход → выход	Исключения
stdin_is_console (static)	определяет, идёт ли ввод с клавиатуры или из файла	→ bool	нет
read_text	печатает приглашение и читает строку; при вводе из файла выводит прочитанную строку на экран; убирает пробелы, табуляции и \r по краям	const std::string& prompt → std::string	InputClosed Error при конце ввода
parse_number	разбирает строку из 1–9 цифр в число; при ошибке value не меняется	const std::string& text, size_t& value → bool	нет
is_code	проверяет, что строка имеет длину depth и состоит из символов 0 и 1	const std::string& text, size_t depth → bool	нет
report_bad_number (static)	печатает причину ошибки ввода числа: пустая строка, не целое неотрицательное число или число вне отрезка	const std::string& text, size_t min, size_t max	нет
read_number	повторяет запрос, пока не будет введено число из отрезка [min, max]	const std::string& prompt, size_t min, size_t max → size_t	InputClosed Error

Имя	Назначение	Вход → выход	Исключения
read_number_or_empty	то же, но пустая строка допустима	const std::string& prompt, size_t min, size_t max, size_t& value → bool: false при пустой строке, value не меняется	InputClosedError
read_letter	повторяет запрос, пока не будет введена одна буква из letters; прописные буквы приводятся к строчным	const std::string& prompt, const std::string& letters, bool allow_empty → char; '\0' при пустой строке, если allow_empty	InputClosedError

Таблица 9 — Методы класса Menu

Имя	Назначение	Вход → выход	Исключения
Menu (конструктор)	создаёт пустой носитель ($n = 0$) и пустые U, A, B; все флаги готовности сброшены	—	нет
run	основной цикл: печатает список команд, читает команду, проверяет её через is_available и вызывает нужный метод; команда q завершает цикл	—	InputClosedError
is_available	проверяет, можно ли выполнить команду в текущем состоянии; если нельзя, печатает, чего не хватает и какой командой это задать	char command → bool	нет
create_universe	запрашивает n и k (при пустой строке k выбирается случайно), строит новый носитель и универсум, сбрасывает A и B, печатает до 16 слов кода и универсум	—	InputClosedError
fill_multiset	запрашивает мощность (при пустой строке — случайная) и способ заполнения; заполняет копию случайно или вручную и только после успеха записывает её в мультимножество	Multiset& multiset, bool& ready, const std::string& name	InputClosedError
fill_manual	читает строки пар, пока мощность target не станет равной cardinality; пустая строка отменяет ввод	Multiset& target, size_t cardinality, const std::string& name → bool: false при отмене	InputClosedError
parse_manual_line	разбирает строку пар код:кратность, проверяет каждую пару и сумму кратностей; при ошибке печатает её причину	const std::string& line, const Multiset& target, size_t left, std::vector<size_t>& indices, std::vector<size_t>& counts → bool	нет
compute_operations	вычисляет 13 операций над A, B и U в порядке массива OPERATIONS	→ std::vector<Multiset>	нет
show_help	печатает формат записи мультимножества, формулы 13 операций и допустимые значения n, k, $\|A\|$, $\|B\|$	—	нет

Имя	Назначение	Вход → выход	Исключения
`show_table`	печатает таблицу кратностей kU, kA, kB для первых `TABLE_LIMIT` = 32 слов и строку Σ с мощностями; у незаданного мультимножества ставится «-»	—	нет
`show_operations`	печатает 13 результатов с мощностями и две проверки: $\lvert\overline{A}\rvert = \lvert U\rvert - \lvert A\rvert$ и $\lvert A \cup B\rvert + \lvert A \cap B\rvert = \lvert A\rvert + \lvert B\rvert$	—	нет
`show_matrix`	печатает матрицу кратностей U, A, B и 13 результатов; столбцов столько, сколько помещается в `MATRIX_WIDTH` = 112 символов, последний столбец — мощности по всем словам	—	нет

Таблица 10 — Служебные функции файла `Menu.cpp` и функция `main`

Имя	Назначение	Вход → выход	Исключения
`text_width` (static)	считает ширину строки в символах: в UTF-8 учитываются только первые байты символов	`const std::string& text` → `size_t`	нет
`pad_right` (static)	дополняет строку пробелами справа до ширины width; длинная строка не обрезается	`const std::string& text`, `size_t width` → `std::string`	нет
`pad_left` (static)	дополняет строку пробелами слева до ширины width	`const std::string& text`, `size_t width` → `std::string`	нет
`format_multiset` (static)	строит запись мультимножества: при $n \leqslant 4$ полностью, при $n > 4$ столько элементов, сколько помещается примерно в `LINE_WIDTH` = 100 символов, но не меньше 4 и не больше 16	`const Multiset& multiset` → `std::string`	нет
`compact` (static)	убирает пробелы из обозначения операции для заголовков матрицы (`A ∪ B` → `A∪B`)	`const std::string& notation` → `std::string`	нет
`main`	включает UTF-8 в консоли Windows, задаёт зерно (`--seed N` или `make_seed`) и печатает его, запускает `Menu::run`	`int argc`, `char* argv[]` → `int`: 0 или 1	перехватывает `InputClosedError` и `std::exception`

Генерация кода Грея показана в листинге 1, автоматическое заполнение — в листинге 2.

Листинг 1 — Генерация кода Грея (`GrayCode.cpp`)

```
static size_t changed_bit(size_t i){
    size_t q = 0;
    size_t j = i;
    while(j % 2 == 0){
        j /= 2;
        q++;
    }
    return q;
```

```
}

std::vector<size_t> generate_gray_code(size_t depth){
    if(depth > MAX_TYPE_DEPTH){
        throw InvalidDepthError();
    }
    std::vector<size_t> codes;
    if(depth == 0){
        return codes;
    }
    size_t count = size_t(1) << depth;
    codes.reserve(count);
    size_t code = 0;
    codes.push_back(code);
    for(size_t i = 1; i < count; i++){
        size_t p = changed_bit(i);
        code ^= size_t(1) << p;
        codes.push_back(code);
    }
    return codes;
}
```

Листинг 2 — Автоматическое заполнение (`Multiset.cpp`)

```
Multiset random_submultiset(const Multiset& bound, size_t cardinality){
    if(cardinality > bound.cardinality()){
        throw CardinalityOverflowError();
    }
    Multiset res(bound.carrier);
    std::vector<size_t> free;
    free.reserve(bound.counts.size());
    for(size_t i = 0; i < bound.counts.size(); i++){
        if(bound.counts[i] > 0) free.push_back(i);
    }
    for(size_t step = 0; step < cardinality; step++){
        size_t j = random_below(free.size());
        size_t i = free[j];
        res.counts[i]++;
        if(res.counts[i] == bound.counts[i]){
            free[j] = free.back();
            free.pop_back();
        }
    }
    return res;
}
```

2.5 Ограничения программы

Ограничения заданы константами. `MAX_DEPTH` = 19 — наибольшая разрядность. `MAX_MULTIPLICITY` = 100 — наибольшая кратность элемента универсума. `MAX_TYPE_DEPTH` = `sizeof(size_t) * 8 - 1` — предел, при котором 2^n ещё помещается в `size_t`; в 32-разрядной сборке он равен 31. Через

меню ввести $n > 19$ нельзя, поэтому исключение `InvalidDepthError` при работе с меню не возникает. Ограничения вывода заданы в `Menu.cpp`: `TABLE_LIMIT` $= 32$ строк таблицы, `PRINT_LIMIT` $= 16$ элементов мультимножества и слов кода, `LINE_WIDTH` $= 100$ и `MATRIX_WIDTH` $= 112$ символов. Числа вводятся не длиннее 9 цифр.

Значение `MAX_DEPTH` выбрано по замеру времени программой `bench` (листинг 17). Она для каждого n от 10 до 21 строит носитель, универсум с наибольшей кратностью 100, заполняет A с мощностью $|U|$ (худший случай заполнения) и выполняет все 13 операций, каждое действие трижды, и берёт наименьшее время. Самое долгое действие в таблице 11 — автоматическое заполнение: оно делает $|U| = 100 \cdot 2^n$ шагов. При $n = 19$ оно занимает около 0,5 с, а при $n = 20$ уже больше 1 с, поэтому выбрано $n \leqslant 19$. Шаг таймера `std::chrono::steady_clock` в этой сборке равен примерно 0,5 мс, поэтому значения меньше 1 мс в таблице измерены грубо.

Таблица 11 — Время действий в худшем случае, мс

n	Код Грея	Универсум	Заполнение A	13 операций	Наибольшее
10	0,0	0,0	0,0	0,0	0,0
11	0,0	0,0	1,0	0,0	1,0
12	0,0	0,0	2,5	0,0	2,5
13	0,0	0,0	5,0	0,0	5,0
14	0,0	0,0	10,0	0,0	10,0
15	0,0	0,0	21,0	1,0	21,0
16	0,0	0,0	44,1	2,0	44,1
17	0,0	0,0	88,5	4,0	88,5
18	0,0	0,0	234,9	7,5	234,9
19	1,3	1,0	532,2	24,0	532,2
20	3,0	3,0	2741,3	48,0	2741,3
21	7,0	7,0	9408,9	105,1	9408,9

Наибольшая кратность 100 выбрана вместе с `MAX_DEPTH`. Автоматическое заполнение делает $m \leqslant |U| = k \cdot 2^n$ шагов, поэтому в худшем случае $m = |U|$ его время растёт пропорционально k; замер в таблице 11 проводился именно для этого случая при $k = 100$. Значит, при $n \leqslant 19$ и $k \leqslant 100$ любое действие занимает меньше 1 с, а при большей кратности, например $k = 200$, заполнение при $n = 19$ заняло бы, по оценке, около 1 с или больше. С вводом чисел этот предел тоже согласован: при $n = 19$ и $k = 100$ мощность универсума равна $100 \cdot 2^{19} = 52\,428\,800 < 999\,999\,999$, поэтому любую допустимую

мощность можно ввести числом не длиннее 9 цифр. Произведение кратностей не больше $100 \cdot 100 = 10\,000$, а сумма мощностей при проверке в `show_operations` не больше $2 \cdot 52\,428\,800$, что меньше наибольшего значения 32-разрядного `size_t`, равного $2^{32} - 1 = 4\,294\,967\,295$; поэтому переполнения нет и в 32-разрядной сборке.

2.6 Обработка ошибок и защита ввода

Весь ввод идёт через функцию `read_text`: она читает строку целиком и убирает пробелы, табуляции и символы `\r` по краям. Если ввод закончился (конец файла или Ctrl+Z), выбрасывается `InputClosedError`; функция `main` перехватывает его, выводит «ввод закончился» и завершает программу с кодом 0.

Числа разбирает `parse_number`: допускается только строка из 1–9 цифр, поэтому знак минус, буквы, пробелы внутри числа и слишком длинные числа отклоняются, а переполнение невозможно. Функции `read_number` и `read_number_or_empty` повторяют запрос, пока не будет введено число из нужного отрезка, а функция `report_bad_number` сообщает одну из трёх причин: пустая строка, не целое неотрицательное число, число вне отрезка. Команды меню и способ заполнения читает `read_letter`: строка должна состоять из одной допустимой буквы, регистр не важен.

Перед выполнением команды метод `is_available` проверяет состояние: без универсума доступны только `u`, `h` и `q`, а операции и матрица требуют, чтобы были заданы оба мультимножества. Сообщение называет, чего не хватает и какой командой это задать.

При ручном вводе метод `parse_manual_line` проверяет каждую пару по порядку: код должен состоять из n символов 0 и 1; после двоеточия должна стоять кратность от 1; кратность не должна превышать свободное место у этого кода с учётом уже введённых раньше и в этой же строке единиц; сумма кратностей строки не должна превышать остаток мощности. При любой ошибке строка отбрасывается целиком. Заполнение идёт в копию мультимножества, поэтому при отмене прежнее мультимножество сохраняется.

Исключения ядра защищают классы от неправильного использования; меню проверяет ввод заранее, поэтому при работе с ним они не возникают, а если возникнут, `main` выведет «Внутренняя ошибка:» и вернёт код 1.

3 Результаты работы

Программа запускалась с вводом из текстовых файлов и параметром `--seed`, поэтому все запуски воспроизводимы: номер сценария совпадает с зерном. При вводе из файла программа сама печатает прочитанную строку после приглашения, поэтому на рисунках виден весь ввод. В выводе программы дополнение $\overline{A}$ печатается как `¬A`, а произведение $A \cdot B$ — как `A * B`. Каждый рисунок показывает одно или несколько связанных действий.

3.1 Корректная работа при $n = 3$

Проверяется основной путь работы: справка, создание универсума при $n = 3$ и $k = 3$, автоматическое заполнение A, ручное заполнение B, таблица кратностей, операции и матрица. Сначала выводится справка с формулами операций и допустимыми значениями (рисунок 1); формулы совпадают с таблицей 1.

```
seed = 1

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> h
запись {000×2, 011×1}: код Грея × кратность, {} - пустое; kU, kA, kB - кратности в U, A, B
A ∪ B  max(kA, kB)               объединение
A ∩ B  min(kA, kB)               пересечение
A \ B  min(kA, kU - kB)          разность
B \ A  min(kB, kU - kA)          разность
A Δ B  max(k[A\B], k[B\A])       симметрическая разность
¬A     kU - kA                   дополнение
¬B     kU - kB                   дополнение
A + B  min(kA + kB, kU)          арифметическая сумма
A - B  max(kA - kB, 0)           арифметическая разность
B - A  max(kB - kA, 0)           арифметическая разность
A * B  min(kA * kB, kU)          арифметическое произведение
A / B  kA div kB; 0 при kB = 0   арифметическое деление
B / A  kB div kA; 0 при kA = 0   арифметическое деление
0 <= n <= 19, 1 <= k <= 100, 0 <= |A|, |B| <= |U|
```

Рисунок 1 — Зерно, список команд и справка

После ввода $n = 3$ и $k = 3$ программа выводит 8 слов кода Грея и универсум мощности $3 \cdot 8 = 24$ (рисунок 2). Порядок слов совпадает с пошаговым построением в таблице 2.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 3
k [1..100, Enter - случайно]: 3
код Грея, 2^3 = 8: 000 001 011 010 110 111 101 100
U = {000×3, 001×3, 011×3, 010×3, 110×3, 111×3, 101×3, 100×3}, |U| = 24
```

Рисунок 2 — Создание универсума при $n = 3$, $k = 3$

На рисунке 3 мультимножество A мощности 10 заполнено автоматически: сумма кратностей $1 + 3 + 1 + 1 + 1 + 3 = 10$, ни одна кратность не больше 3.

Мультимножество B мощности 8 введено вручную двумя строками; код `110` без двоеточия получил кратность 1.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..24, Enter - случайно]: 10
r - случайно, m - вручную, Enter - отмена: r
A = {000×1, 001×3, 011×1, 010×1, 111×1, 100×3}, |A| = 10

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..24, Enter - случайно]: 8
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 3, Enter - отмена
B = {}, осталось 8: 000:2 011:3 110
B = {000×2, 011×3, 110×1}, осталось 2: 101:2
B = {000×2, 011×3, 110×1, 101×2}, |B| = 8
```

Рисунок 3 — Автоматическое заполнение A ($|A| = 10$) и ручное заполнение B ($|B| = 8$)

Таблица кратностей (рисунок 4) показывает $k_U(x), k_A(x), k_B(x)$ для каждого слова и мощности 24, 10 и 8 в строке Σ.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код  kU  kA  kB
000   3   1   2
001   3   3   0
011   3   1   3
010   3   1   0
110   3   0   1
111   3   1   0
101   3   0   2
100   3   3   0
Σ    24  10   8
```

Рисунок 4 — Таблица кратностей

Результаты всех 13 операций показаны на рисунке 5, а матрица кратностей — на рисунке 6. Обе проверки, которые выводит программа, выполнены: $14 = 24 - 10$ и $16 + 2 = 10 + 8$. Видно также, что $|A \triangle B| = 15$, а не $|A \cup B| - |A \cap B| = 14$: это следствие формулы симметрической разности, описанной в разделе 1.2.

Ручная проверка проведена для слов `000` ($k_U(x) = 3, k_A(x) = 1, k_B(x) = 2$), `001` $(3, 3, 0)$ и `011` $(3, 1, 3)$ по формулам таблицы 1. Результаты приведены в таблице 12; в последнем столбце — значения из столбцов `000`, `001`, `011` матрицы на рисунке 6. Все значения совпали.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {000×2, 001×3, 011×3, 010×1, 110×1, 111×1, 101×2, 100×3}, |A ∪ B| = 16
A ∩ B = {000×1, 011×1}, |A ∩ B| = 2
A \ B = {000×1, 001×3, 010×1, 111×1, 100×3}, |A \ B| = 9
B \ A = {000×2, 011×2, 110×1, 101×2}, |B \ A| = 7
A Δ B = {000×2, 001×3, 011×2, 010×1, 110×1, 111×1, 101×2, 100×3}, |A Δ B| = 15
¬A    = {000×2, 011×2, 010×2, 110×3, 111×2, 101×3}, |¬A| = 14
¬B    = {000×1, 001×3, 010×3, 110×2, 111×3, 101×1, 100×3}, |¬B| = 16
A + B = {000×3, 001×3, 011×3, 010×1, 110×1, 111×1, 101×2, 100×3}, |A + B| = 17
A - B = {001×3, 010×1, 111×1, 100×3}, |A - B| = 8
B - A = {000×1, 011×2, 110×1, 101×2}, |B - A| = 6
A * B = {000×2, 011×3}, |A * B| = 5
A / B = {}, |A / B| = 0
B / A = {000×2, 011×3}, |B / A| = 5
проверка: |¬A| = |U| - |A|: 14 = 24 - 10 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 16 + 2 = 10 + 8 верно
```

Рисунок 5 — Результаты операций и проверки

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> m
      000  001  011  010  110  111  101  100      |·|
U       3    3    3    3    3    3    3    3       24
A       1    3    1    1    0    1    0    3       10
B       2    0    3    0    1    0    2    0        8
A∪B     2    3    3    1    1    1    2    3       16
A∩B     1    0    1    0    0    0    0    0        2
A\B     1    3    0    1    0    1    0    3        9
B\A     2    0    2    0    1    0    2    0        7
AΔB     2    3    2    1    1    1    2    3       15
¬A      2    0    2    2    3    2    3    0       14
¬B      1    3    0    3    2    3    1    3       16
A+B     3    3    3    1    1    1    2    3       17
A-B     0    3    0    1    0    1    0    3        8
B-A     1    0    2    0    1    0    2    0        6
A*B     2    0    3    0    0    0    0    0        5
A/B     0    0    0    0    0    0    0    0        0
B/A     2    0    3    0    0    0    0    0        5

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> q
```

Рисунок 6 — Матрица кратностей и выход

Таблица 12 — Ручная проверка операций для трёх слов

Опер.	**000**	**001**	**011**	**Вывод**
$A \cup B$	$\max(1,2) = 2$	$\max(3,0) = 3$	$\max(1,3) = 3$	2, 3, 3
$A \cap B$	$\min(1,2) = 1$	$\min(3,0) = 0$	$\min(1,3) = 1$	1, 0, 1
$A \setminus B$	$\min(1, 3-2) = 1$	$\min(3, 3-0) = 3$	$\min(1, 3-3) = 0$	1, 3, 0
$B \setminus A$	$\min(2, 3-1) = 2$	$\min(0, 3-3) = 0$	$\min(3, 3-1) = 2$	2, 0, 2
$A \triangle B$	$\max(1,2) = 2$	$\max(3,0) = 3$	$\max(0,2) = 2$	2, 3, 2
$\overline{A}$	$3-1 = 2$	$3-3 = 0$	$3-1 = 2$	2, 0, 2
$\overline{B}$	$3-2 = 1$	$3-0 = 3$	$3-3 = 0$	1, 3, 0
$A + B$	$\min(1+2, 3) = 3$	$\min(3+0, 3) = 3$	$\min(1+3, 3) = 3$	3, 3, 3
$A - B$	$\max(1-2, 0) = 0$	$\max(3-0, 0) = 3$	$\max(1-3, 0) = 0$	0, 3, 0
$B - A$	$\max(2-1, 0) = 1$	$\max(0-3, 0) = 0$	$\max(3-1, 0) = 2$	1, 0, 2
$A \cdot B$	$\min(1 \cdot 2, 3) = 2$	$\min(3 \cdot 0, 3) = 0$	$\min(1 \cdot 3, 3) = 3$	2, 0, 3
A/B	$1 \operatorname{div} 2 = 0$	$k_B(x) = 0 \Rightarrow 0$	$1 \operatorname{div} 3 = 0$	0, 0, 0
B/A	$2 \operatorname{div} 1 = 2$	$0 \operatorname{div} 3 = 0$	$3 \operatorname{div} 1 = 3$	2, 0, 3

3.2 Некорректный ввод команд и параметров

Проверяется реакция на команды, недоступные в текущем состоянии, и на неверный ввод в меню и при вводе чисел. Пока универсум не создан, команды `a` и `o` не выполняются, и программа подсказывает команду `u` (рисунок 7).

```
seed = 2

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
! команда a недоступна: универсум ещё не создан (команда u)

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
! команда o недоступна: универсум ещё не создан (команда u)
```

Рисунок 7 — Команды до создания универсума

На рисунке 8 в меню введены строка из трёх букв, цифра, пустая строка, повторённая буква и отрицательное число. Каждый раз выводится причина и повторяется приглашение. Прописная буква `U` принята как команда `u`.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> abc
! «abc» не команда; допустимы буквы u a b t o m h q
> 7
! «7» не команда; допустимы буквы u a b t o m h q
>
! пустая строка: введите одну из букв u a b t o m h q
> uu
! «uu» не команда; допустимы буквы u a b t o m h q
> -5
! «-5» не команда; допустимы буквы u a b t o m h q
> U
```

Рисунок 8 — Неверный ввод команды меню

При вводе разрядности отклонены значения `-1`, `20`, `abc`, `3x` и пустая строка, при вводе кратности — `0`, `101` и `-5` (рисунок 9). Сообщения различают три причины: не число, число вне отрезка и пустая строка.

На рисунке 10 после создания универсума команды `o` и `m` всё ещё недоступны, и программа перечисляет, какие мультимножества не заданы. Затем при вводе мощности A отклонены число 9 (больше $|U| = 8$) и буква, при выборе способа заполнения — буква `z`. Пустая строка отменила заполнение, и команда `o` по-прежнему недоступна.

```
n [0..19]: -1
! «-1» не целое неотрицательное число; нужно число из [0..19]
n [0..19]: 20
! 20 не входит в [0..19]
n [0..19]: abc
! «abc» не целое неотрицательное число; нужно число из [0..19]
n [0..19]: 3x
! «3x» не целое неотрицательное число; нужно число из [0..19]
n [0..19]:
! пустая строка: нужно целое число из [0..19]
n [0..19]: 2
k [1..100, Enter - случайно]: 0
! 0 не входит в [1..100]
k [1..100, Enter - случайно]: 101
! 101 не входит в [1..100]
k [1..100, Enter - случайно]:  -5
! «-5» не целое неотрицательное число; нужно число из [1..100]
k [1..100, Enter - случайно]: 2
код Грея, 2^2 = 4: 00 01 11 10
U = {00×2, 01×2, 11×2, 10×2}, |U| = 8
```

Рисунок 9 — Неверный ввод разрядности и кратности

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
! команда o недоступна: не задано A (команда a) и B (команда b)

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> m
! команда m недоступна: не задано A (команда a) и B (команда b)

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..8, Enter - случайно]: 9
! 9 не входит в [0..8]
|A| [0..8, Enter - случайно]: x
! «x» не целое неотрицательное число; нужно число из [0..8]
|A| [0..8, Enter - случайно]: 4
r - случайно, m - вручную, Enter - отмена: z
! «z» не команда; допустимы буквы r m
r - случайно, m - вручную, Enter - отмена:
отменено

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
! команда o недоступна: не задано A (команда a) и B (команда b)

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> q
```

Рисунок 10 — Недоступные команды, неверная мощность, неверный способ заполнения и отмена

3.3 Ошибки при ручном заполнении

Проверяется разбор строк ручного ввода. Создан универсум при $n = 2$, $k = 2$, и автоматически заполнено B мощности 3 (рисунок 11).

```
seed = 3

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 2
k [1..100, Enter - случайно]: 2
код Грея, 2^2 = 4: 00 01 11 10
U = {00×2, 01×2, 11×2, 10×2}, |U| = 8

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..8, Enter - случайно]: 3
r - случайно, m - вручную, Enter - отмена: r
B = {00×1, 11×2}, |B| = 3
```

Рисунок 11 — Подготовка: универсум $n = 2$, $k = 2$ и мультимножество B

На рисунке 12 заполняется A мощности 5. Отклонены: код неверной длины `0:1`, код с буквой `0a`, нечисловая кратность `00:x`, нулевая кратность `00:0`, кратность больше k `00:3`, повтор кода в одной строке `00:2 00` и строка с суммой 6 при остатке 5. Строка `00:2 01` принята, после неё повторный ввод кода `00` отклонён, так как у него не осталось места. Пустая строка отменила заполнение.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..8, Enter - случайно]: 5
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
A = {}, осталось 5: 0:1
! «0:1»: код должен состоять из 2 символов 0 и 1
A = {}, осталось 5: 0a
! «0a»: код должен состоять из 2 символов 0 и 1
A = {}, осталось 5: 00:x
! «00:x»: после двоеточия нужна кратность - целое число от 1
A = {}, осталось 5: 00:0
! «00:0»: после двоеточия нужна кратность - целое число от 1
A = {}, осталось 5: 00:3
! «00:3»: у 00 свободно 2 (kU = 2, занято 0)
A = {}, осталось 5: 00:2 00
! «00»: у 00 свободно 0 (kU = 2, занято 2)
A = {}, осталось 5: 00:2 01:2 11:2
! в строке 6 ед., а осталось распределить 5
A = {}, осталось 5: 00:2 01
A = {00×2, 01×1}, осталось 2: 00
! «00»: у 00 свободно 0 (kU = 2, занято 2)
A = {00×2, 01×1}, осталось 2:
отменено
```

Рисунок 12 — Ошибки ручного ввода и отмена

После отмены A осталось незаданным: в таблице на рисунке 13 в столбце `kA` стоят прочерки, а частично введённые значения не сохранились.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код  kU  kA  kB
00    2   -   1
01    2   -   0
11    2   -   2
10    2   -   0
Σ     8   -   3
```

Рисунок 13 — Таблица после отмены заполнения A

Затем A мощности 3 введено одной строкой, а ручной ввод B мощности 4 отменён пустой строкой (рисунок 14). Таблица в конце рисунка показывает, что прежнее B с кратностями $1, 0, 2, 0$ сохранилось.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..8, Enter - случайно]: 3
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
A = {}, осталось 3: 11:2 10
A = {11×2, 10×1}, |A| = 3

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..8, Enter - случайно]: 4
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
B = {}, осталось 4:
отменено

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код  kU  kA  kB
00    2   0   1
01    2   0   0
11    2   2   2
10    2   1   0
Σ     8   3   3

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> q
```

Рисунок 14 — Успешное заполнение A и отмена ввода B с сохранением прежнего значения

3.4 Граничные значения мощности и кратности

Проверяются мощность, равная мощности универсума, нулевая мощность, наименьшая и наибольшая кратность. Создан универсум при $n = 2$ с наименьшей кратностью $k = 1$, A заполнено автоматически с мощностью $|U| = 4$, B — с мощностью 0 (рисунок 15). При $|A| = |U|$ автоматическое заполнение даёт $A = U$.

```
seed = 4

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 2
k [1..100, Enter - случайно]: 1
код Грея, 2^2 = 4: 00 01 11 10
U = {00×1, 01×1, 11×1, 10×1}, |U| = 4

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..4, Enter - случайно]: 4
r - случайно, m - вручную, Enter - отмена: r
A = {00×1, 01×1, 11×1, 10×1}, |A| = 4

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..4, Enter - случайно]: 0
r - случайно, m - вручную, Enter - отмена: r
B = {}, |B| = 0
```

Рисунок 15 — Универсум с $k = 1$, $|A| = |U|$ и $|B| = 0$

Результаты операций (рисунок 16) соответствуют формулам: $\overline{A}$ пусто, $\overline{B} = U$, $A \cup B = A \setminus B = A - B = U$, пересечение и произведение пусты. Деление A/B даёт пустое мультимножество, так как при $k_B(x) = 0$ результат равен нулю.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {00×1, 01×1, 11×1, 10×1}, |A ∪ B| = 4
A ∩ B = {}, |A ∩ B| = 0
A \ B = {00×1, 01×1, 11×1, 10×1}, |A \ B| = 4
B \ A = {}, |B \ A| = 0
A Δ B = {00×1, 01×1, 11×1, 10×1}, |A Δ B| = 4
¬A    = {}, |¬A| = 0
¬B    = {00×1, 01×1, 11×1, 10×1}, |¬B| = 4
A + B = {00×1, 01×1, 11×1, 10×1}, |A + B| = 4
A - B = {00×1, 01×1, 11×1, 10×1}, |A - B| = 4
B - A = {}, |B - A| = 0
A * B = {}, |A * B| = 0
A / B = {}, |A / B| = 0
B / A = {}, |B / A| = 0
проверка: |¬A| = |U| - |A|: 0 = 4 - 4 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 4 + 0 = 4 + 0 верно
```

Рисунок 16 — Операции при $A = U$ и $B = \emptyset$

Далее универсум пересоздан при $n = 1$ с наибольшей кратностью $k = 100$, мощность $|U| = 200$; программа сообщила, что A и B сброшены. Мультимножество A мощности 200 введено вручную одной строкой `0:100 1:100`, а при ручном заполнении B с мощностью 0 строки не запрашивались (рисунок 17). Операции для этого случая показаны на рисунке 18.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 1
k [1..100, Enter - случайно]: 100
код Грея, 2^1 = 2: 0 1
U = {0×100, 1×100}, |U| = 200
A и B сброшены

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..200, Enter - случайно]: 200
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 100, Enter - отмена
A = {}, осталось 200: 0:100 1:100
A = {0×100, 1×100}, |A| = 200

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..200, Enter - случайно]: 0
r - случайно, m - вручную, Enter - отмена: m
B = {}, |B| = 0
```

Рисунок 17 — Универсум с $k = 100$ и ручной ввод при $|A| = |U|$ и $|B| = 0$

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {0×100, 1×100}, |A ∪ B| = 200
A ∩ B = {}, |A ∩ B| = 0
A \ B = {0×100, 1×100}, |A \ B| = 200
B \ A = {}, |B \ A| = 0
A Δ B = {0×100, 1×100}, |A Δ B| = 200
¬A    = {}, |¬A| = 0
¬B    = {0×100, 1×100}, |¬B| = 200
A + B = {0×100, 1×100}, |A + B| = 200
A - B = {0×100, 1×100}, |A - B| = 200
B - A = {}, |B - A| = 0
A * B = {}, |A * B| = 0
A / B = {}, |A / B| = 0
B / A = {}, |B / A| = 0
проверка: |¬A| = |U| - |A|: 0 = 200 - 200 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 200 + 0 = 200 + 0 верно

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> q
```

Рисунок 18 — Операции при $k = 100$

3.5 Пустой универсум, пересоздание и конец ввода

Проверяется разрядность $n = 0$, при которой программа по соглашению из подраздела 1.1 считает код Грея пустым. Кратность в этом случае не запрашивается, универсум пуст, допустимая мощность A и B — только 0, а при пустой строке случайная мощность тоже равна 0 (рисунок 19).

```
seed = 5

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 0
при n = 0 в коде Грея нет слов, кратность не нужна
U = {}, |U| = 0

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код  kU  kA  kB
Σ     0   -   -

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..0, Enter - случайно]: 0
r - случайно, m - вручную, Enter - отмена: r
A = {}, |A| = 0

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..0, Enter - случайно]:
|B| = 0
r - случайно, m - вручную, Enter - отмена: m
B = {}, |B| = 0
```

Рисунок 19 — Универсум при $n = 0$ и пустые A, B

Все операции дают пустые мультимножества, проверки выполнены (рисунок 20). Матрица при $n = 0$ не содержит ни одного столбца с кодом, в ней выводятся только нулевые мощности.

Затем универсум пересоздан при $n = 4$ с пустой строкой вместо кратности: программа выбрала $k = 6$, вывела все 16 слов кода, $|U| = 96$, и сообщила о сбросе A и B (рисунок 21). В следующей за этим таблице у A и B стоят

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {}, |A ∪ B| = 0
A ∩ B = {}, |A ∩ B| = 0
A \ B = {}, |A \ B| = 0
B \ A = {}, |B \ A| = 0
A Δ B = {}, |A Δ B| = 0
¬A    = {}, |¬A| = 0
¬B    = {}, |¬B| = 0
A + B = {}, |A + B| = 0
A - B = {}, |A - B| = 0
B - A = {}, |B - A| = 0
A * B = {}, |A * B| = 0
A / B = {}, |A / B| = 0
B / A = {}, |B / A| = 0
проверка: |¬A| = |U| - |A|: 0 = 0 - 0 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 0 + 0 = 0 + 0 верно
```

Рисунок 20 — Операции при $n = 0$

прочерки. После неё входной файл закончился без команды q, и программа завершилась сообщением «ввод закончился».

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 4
k [1..100, Enter - случайно]:
k = 6
код Грея, 2^4 = 16: 0000 0001 0011 0010 0110 0111 0101 0100 1100 1101 1111 1110 1010 1011 1001 1000
U = {0000×6, 0001×6, 0011×6, 0010×6, 0110×6, 0111×6, 0101×6, 0100×6, 1100×6, 1101×6, 1111×6, 1110×6, 1010×6, 101
1×6, 1001×6, 1000×6}, |U| = 96
A и B сброшены

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код   kU  kA  kB
0000   6   -   -
0001   6   -   -
0011   6   -   -
0010   6   -   -
0110   6   -   -
0111   6   -   -
0101   6   -   -
0100   6   -   -
1100   6   -   -
1101   6   -   -
1111   6   -   -
1110   6   -   -
1010   6   -   -
1011   6   -   -
1001   6   -   -
1000   6   -   -
Σ     96   -   -

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
>
ввод закончился
```

Рисунок 21 — Пересоздание универсума со случайной кратностью, таблица после сброса и конец ввода

3.6 Наибольшая разрядность

Проверяется работа при $n = 19$ и ограничения вывода. Код Грея содержит $2^{19} = 524\,288$ слов; программа выводит первые 16 и число остальных, мощность универсума при $k = 5$ равна 2 621 440 (рисунок 22).

Мощность A выбрана случайно и равна 2 341 150, мощность B задана числом 1 000 000 (рисунок 23). В записи мультимножеств показаны только

```
seed = 6

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> u
n [0..19]: 19
k [1..100, Enter - случайно]: 5
код Грея, 2^19 = 524288:
0000000000000000000 0000000000000000001 0000000000000000011 0000000000000000010 0000000000000000110
0000000000000000111 0000000000000000101 0000000000000000100 0000000000000001100 0000000000000001101
0000000000000001111 0000000000000001110 0000000000000001010 0000000000000001011 0000000000000001001
0000000000000001000
... и ещё 524272
U = {0000000000000000000×5, 0000000000000000001×5, 0000000000000000011×5, 0000000000000000010×5, ... ещё 524284
эл.}, |U| = 2621440
```

Рисунок 22 — Универсум при $n = 19$, $k = 5$

первые элементы и число скрытых.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> a
|A| [0..2621440, Enter - случайно]:
|A| = 2341150
r - случайно, m - вручную, Enter - отмена: r
A = {0000000000000000000×2, 0000000000000000001×5, 0000000000000000011×5, 0000000000000000010×5, ... ещё 522901
эл.}, |A| = 2341150

u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> b
|B| [0..2621440, Enter - случайно]: 1000000
r - случайно, m - вручную, Enter - отмена: r
B = {0000000000000000000×2, 0000000000000000001×2, 0000000000000000011×1, 0000000000000000110×1, ... ещё 448234
эл.}, |B| = 1000000
```

Рисунок 23 — Автоматическое заполнение A и B при $n = 19$

Таблица кратностей ограничена 32 строками, но строка Σ содержит мощности по всем элементам (рисунок 24).

Обе проверки на рисунке 25 выполнены: $280\,290 = 2\,621\,440 - 2\,341\,150$ и $2\,376\,201 + 964\,949 = 2\,341\,150 + 1\,000\,000$. Матрица (рисунок 26) показывает 4 столбца, которые помещаются в ширину 112 символов, а мощности считаются по всем словам. После неё ввод $n = 20$ отклонён, так как это больше `MAX_DEPTH`, и ввод закончился.

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> t
код                     kU      kA      kB
0000000000000000000      5       2       2
0000000000000000001      5       5       2
0000000000000000011      5       5       1
0000000000000000010      5       5       0
0000000000000000110      5       4       1
0000000000000000111      5       5       1
0000000000000000101      5       5       4
0000000000000000100      5       4       2
0000000000000001100      5       3       1
0000000000000001101      5       5       1
0000000000000001111      5       5       2
0000000000000001110      5       5       3
0000000000000001010      5       5       4
0000000000000001011      5       4       1
0000000000000001001      5       4       2
0000000000000001000      5       5       1
0000000000000011000      5       4       3
0000000000000011001      5       4       2
0000000000000011011      5       5       3
0000000000000011010      5       5       2
0000000000000011110      5       5       2
0000000000000011111      5       5       0
0000000000000011101      5       5       1
0000000000000011100      5       4       1
0000000000000010100      5       4       4
0000000000000010101      5       3       1
0000000000000010111      5       5       2
0000000000000010110      5       3       3
0000000000000010010      5       3       3
0000000000000010011      5       5       1
0000000000000010001      5       5       0
0000000000000010000      5       5       1
... показаны 32 из 524288
Σ                  2621440 2341150 1000000
```

Рисунок 24 — Таблица кратностей при $n = 19$

```
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> o
A ∪ B = {0000000000000000000×2, 0000000000000000001×5, 0000000000000000011×5, 0000000000000000010×5, ... ещё 524
056 эл.}, |A ∪ B| = 2376201
A ∩ B = {0000000000000000000×2, 0000000000000000001×2, 0000000000000000011×1, 0000000000000000110×1, ... ещё 447
079 эл.}, |A ∩ B| = 964949
A \ B = {0000000000000000000×2, 0000000000000000001×3, 0000000000000000011×4, 0000000000000000010×5, ... ещё 498
578 эл.}, |A \ B| = 1530512
B \ A = {0000000000000000000×2, 0000000000000000110×1, 0000000000000000100×1, 0000000000000001100×1, ... ещё 130
946 эл.}, |B \ A| = 189362
A Δ B = {0000000000000000000×2, 0000000000000000001×3, 0000000000000000011×4, 0000000000000000010×5, ... ещё 506
874 эл.}, |A Δ B| = 1565563
¬A    = {0000000000000000000×3, 0000000000000000110×1, 0000000000000000100×1, 0000000000000001100×2, ... ещё 153
341 эл.}, |¬A| = 280290
¬B    = {0000000000000000000×3, 0000000000000000001×3, 0000000000000000011×4, 0000000000000000010×5, ... ещё 499
899 эл.}, |¬B| = 1621440
A + B = {0000000000000000000×4, 0000000000000000001×5, 0000000000000000011×5, 0000000000000000010×5, ... ещё 524
056 эл.}, |A + B| = 2530512
A - B = {0000000000000000001×3, 0000000000000000011×4, 0000000000000000010×5, 0000000000000000110×3, ... ещё 460
829 эл.}, |A - B| = 1376201
B - A = {0000000000000101011×1, 0000000000000100101×3, 0000000000001111000×3, 0000000000001000111×2, ... ещё 230
34 эл.}, |B - A| = 35051
A * B = {0000000000000000000×4, 0000000000000000001×5, 0000000000000000011×5, 0000000000000000110×4, ... ещё 447
079 эл.}, |A * B| = 2141208
A / B = {0000000000000000000×1, 0000000000000000001×2, 0000000000000000011×5, 0000000000000000110×4, ... ещё 425
196 эл.}, |A / B| = 1058008
B / A = {0000000000000000000×1, 0000000000000010100×1, 0000000000000010110×1, 0000000000000010010×1, ... ещё 620
68 эл.}, |B / A| = 73975
проверка: |¬A| = |U| - |A|: 280290 = 2621440 - 2341150 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 2376201 + 964949 = 2341150 + 1000000 верно
```

Рисунок 25 — Операции при $n = 19$

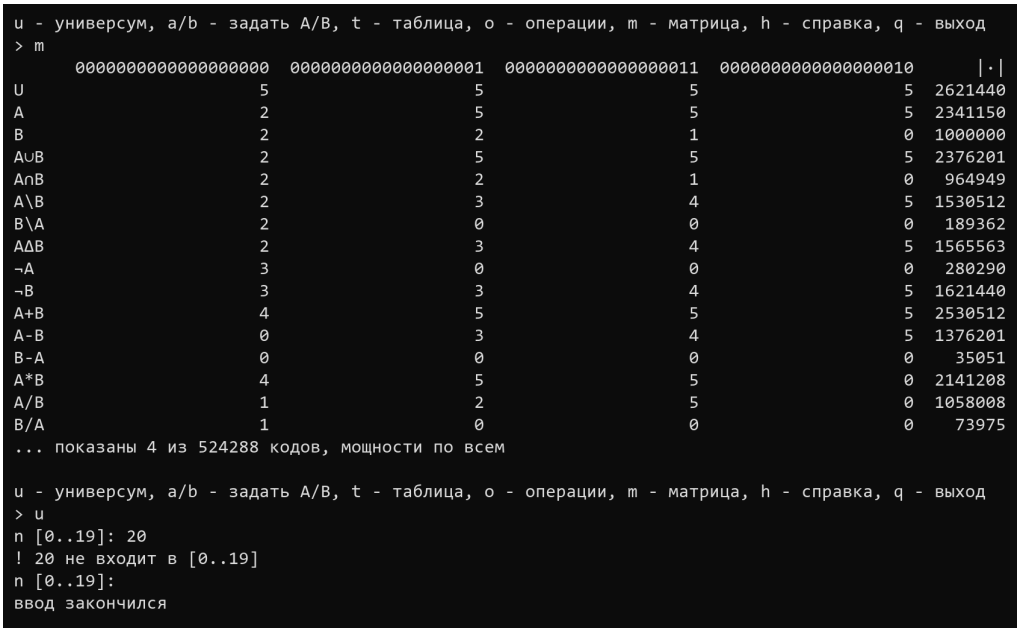


Рисунок 26 — Матрица кратностей при $n = 19$, попытка задать $n = 20$ и конец ввода

ЗАКЛЮЧЕНИЕ

В ходе работы разработана программа для работы с мультимножествами, элементами которых служат слова бинарного кода Грея. В программе реализовано:

– генерация бинарного кода Грея разрядности n от 0 до 19, которую задаёт пользователь, и заполнение его словами универсума мультимножеств с одинаковой кратностью k от 1 до 100; при этих пределах любое действие программы занимает меньше 1 с (таблица 11);
– формирование на основе универсума двух мультимножеств A и B с мощностями, которые задаёт пользователь, причём способ заполнения каждого из них — вручную или автоматически — пользователь выбирает сам;
– вычисление и вывод результатов операций над A и B: объединения, пересечения, разностей $A \setminus B$ и $B \setminus A$, симметрической разности, дополнений $\overline{A}$ и $\overline{B}$, арифметических суммы, разности, произведения и деления; результаты выводятся записью мультимножеств и матрицей кратностей, а две проверки мощностей программа выполняет сама;
– защита от некорректного ввода с указанием причины ошибки и пользовательское меню, в котором недоступные команды объясняют, чего не хватает.

Работа программы проверена на шести сценариях, ручной расчёт операций для трёх слов совпал с выводом программы.

Преимущества:

– код Грея строится без рекурсии за $O(2^n)$, так как алгоритм делает $2^n - 1$ шагов, а общее число проверок при поиске меняемых разрядов меньше $2 \cdot 2^n$ (19);
– номер слова при ручном вводе находится за $O(n)$ по обратной таблице номеров, без просмотра всей последовательности;
– гарантируется математическая замкнутость: результаты всех операций являются подмультимножествами универсума и вычисляются за $O(2^n)$, а автоматическое заполнение за $O(2^n + m)$ даёт мощность ровно m, не превышая кратностей универсума.

Недостатки:

– каждое мультимножество занимает $O(2^n)$ памяти независимо от

мощности, так как хранятся кратности всех слов носителя, включая нулевые;

– основным узким местом является автоматическое заполнение: при трудоёмкости $O(2^n + m)$ и m, близких к $|U| = k \cdot 2^n$, оно занимает около 0,5 с при $n = 19$ и больше 1 с при $n = 20$ (таблица 11), поэтому разрядность ограничена значением 19;
– универсум может быть только равномерным, так как пользователь задаёт одну кратность k для всех слов.

Программа масштабируема: ядро не зависит от интерфейса, поэтому новые возможности добавляются отдельными функциями без изменения ядра. Например:

– новые теоретико-множественные операции конструируются на базе существующих — так уже построены разность и симметрическая разность;
– неравномерный универсум задаётся отдельной функцией, так как мультимножество хранится как произвольная функция кратности;
– другой способ заполнения добавляется отдельной функцией, возвращающей мультимножество.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Новиков, Ф. А. Дискретная математика для программистов : учебник для вузов / Ф. А. Новиков. — 3-е изд. — Санкт-Петербург : Питер, 2009. — 384 с.

Востров, А. В. Дискретная математика : курс лекций / А. В. Востров ; Санкт-Петербургский политехнический университет Петра Великого. — Санкт-Петербург, 2026. — URL: https://tema.spbstu.ru/dismath_lect/ (дата обращения: 04.10.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А
Текст программы

В приложении приведены исходные файлы проекта: файл сборки (листинг 3), главный файл (листинг 4), файлы интерфейса (листинги 5, 6, 7, 8), файлы ядра (листинги 9, 10, 11, 12, 13, 14, 15, 16) и программа замера времени (листинг 17).

Листинг 3 — `CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.10)
project(lab1 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 14)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

set(CORE_SOURCES
    src/core/GrayCode.cpp
    src/core/Random.cpp
    src/core/Multiset.cpp
    src/core/Universe.cpp
)

add_executable(lab1 src/main.cpp src/ui/Input.cpp src/ui/Menu.cpp ${CORE_SOURCES})
target_include_directories(lab1 PRIVATE src)

add_executable(bench tools/bench.cpp ${CORE_SOURCES})
target_include_directories(bench PRIVATE src)
```

Листинг 4 — `src/main.cpp`

```
#include "core/Random.hpp"
#include "ui/Input.hpp"
#include "ui/Menu.hpp"
#include <cstdlib>
#include <cstring>
#include <exception>
#include <iostream>
#ifdef _WIN32
#include <windows.h>
#endif

int main(int argc, char* argv[]){
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif
    unsigned seed = make_seed();
    if(argc == 3 && std::strcmp(argv[1], "--seed") == 0){
        seed = static_cast<unsigned>(std::strtoul(argv[2], nullptr, 10));
    }
    seed_random(seed);
    std::cout << "seed = " << seed << std::endl;
```

```
22
23      try{
24          Menu menu;
25          menu.run();
26      }
27      catch(const InputClosedError&){
28          std::cout << std::endl << "ввод закончился" << std::endl;
29      }
30      catch(const std::exception& error){
31          std::cout << "Внутренняя ошибка: " << error.what() << std::endl;
32          return 1;
33      }
34      return 0;
35  }
```

Листинг 5 — `src/ui/Input.hpp`

```
1   #pragma once
2   #include <cstddef>
3   #include <stdexcept>
4   #include <string>
5
6   class InputClosedError : public std::runtime_error{
7   public:
8       InputClosedError():std::runtime_error("ввод закончился"){}
9   };
10
11  std::string read_text(const std::string& prompt);
12
13  bool parse_number(const std::string& text, size_t& value);
14
15  bool is_code(const std::string& text, size_t depth);
16
17  size_t read_number(const std::string& prompt, size_t min, size_t max);
18
19  bool read_number_or_empty(const std::string& prompt, size_t min, size_t max, size_t&
    value);
20
21  char read_letter(const std::string& prompt, const std::string& letters, bool allow_empty);
```

Листинг 6 — `src/ui/Input.cpp`

```
1   #include "ui/Input.hpp"
2   #include <iostream>
3   #ifdef _WIN32
4   #include <io.h>
5   static bool stdin_is_console(){ return _isatty(0) != 0; }
6   #else
7   #include <unistd.h>
8   static bool stdin_is_console(){ return isatty(0) != 0; }
9   #endif
10
11  std::string read_text(const std::string& prompt){
12      std::cout << prompt;
13      std::string line;
```

```
    if(!std::getline(std::cin, line)){
        throw InputClosedError();
    }
    static const bool echo = !stdin_is_console();
    if(echo) std::cout << line.substr(0, line.find('\r')) << std::endl;
    size_t begin = line.find_first_not_of(" \t\r");
    if(begin == std::string::npos) return "";
    size_t end = line.find_last_not_of(" \t\r");
    return line.substr(begin, end - begin + 1);
}

bool parse_number(const std::string& text, size_t& value){
    if(text.empty() || text.size() > 9) return false;
    size_t res = 0;
    for(size_t i = 0; i < text.size(); i++){
        if(text[i] < '0' || text[i] > '9') return false;
        res = res * 10 + (text[i] - '0');
    }
    value = res;
    return true;
}

bool is_code(const std::string& text, size_t depth){
    if(text.size() != depth) return false;
    for(size_t i = 0; i < text.size(); i++){
        if(text[i] != '0' && text[i] != '1') return false;
    }
    return true;
}

static void report_bad_number(const std::string& text, size_t min, size_t max){
    std::string range = "[" + std::to_string(min) + ".." + std::to_string(max) + "]";
    size_t value = 0;
    if(text.empty()){
        std::cout << "! пустая строка: нужно целое число из " << range << std::endl;
    }
    else if(parse_number(text, value)){
        std::cout << "! " << value << " не входит в " << range << std::endl;
    }
    else{
        std::cout << "! «" << text << "» не целое неотрицательное число; нужно число из "
        << range << std::endl;
    }
}

size_t read_number(const std::string& prompt, size_t min, size_t max){
    while(true){
        std::string line = read_text(prompt);
        size_t value = 0;
        if(parse_number(line, value) && value >= min && value <= max){
            return value;
        }
        report_bad_number(line, min, max);
    }
}

bool read_number_or_empty(const std::string& prompt, size_t min, size_t max, size_t&
value){
    while(true){
        std::string line = read_text(prompt);
        if(line.empty()) return false;
```

```
        size_t res = 0;
        if(parse_number(line, res) && res >= min && res <= max){
            value = res;
            return true;
        }
        report_bad_number(line, min, max);
    }
}

char read_letter(const std::string& prompt, const std::string& letters, bool allow_empty){
    std::string listed;
    for(size_t i = 0; i < letters.size(); i++){
        listed += (i > 0 ? " " : "") + std::string(1, letters[i]);
    }
    while(true){
        std::string line = read_text(prompt);
        if(line.empty() && allow_empty) return '\0';
        if(line.size() == 1){
            char letter = line[0];
            if(letter >= 'A' && letter <= 'Z') letter = letter - 'A' + 'a';
            if(letters.find(letter) != std::string::npos) return letter;
        }
        if(line.empty()){
            std::cout << "! пустая строка: введите одну из букв " << listed << std::endl;
        }
        else{
            std::cout << "! «" << line << "» не команда; допустимы буквы " << listed <<
            std::endl;
        }
    }
}
```

Листинг 7 — `src/ui/Menu.hpp`

```
#pragma once
#include "core/GrayCode.hpp"
#include "core/Multiset.hpp"
#include <cstddef>
#include <memory>
#include <string>
#include <vector>

class Menu{
    std::shared_ptr<const CodeSet> carrier;
    Multiset universe;
    Multiset a;
    Multiset b;
    bool universe_ready;
    bool a_ready;
    bool b_ready;

    bool is_available(char command) const;

    void create_universe();
    void fill_multiset(Multiset& multiset, bool& ready, const std::string& name);
    bool fill_manual(Multiset& target, size_t cardinality, const std::string& name) const;
    bool parse_manual_line(const std::string& line, const Multiset& target, size_t left,
                           std::vector<size_t>& indices, std::vector<size_t>& counts)
                           const;
```

```
25
26      std::vector<Multiset> compute_operations() const;
27      void show_help() const;
28      void show_table() const;
29      void show_operations() const;
30      void show_matrix() const;
31
32  public:
33      Menu();
34      void run();
35  };
```

Листинг 8 — `src/ui/Menu.cpp`

```
1   #include "ui/Menu.hpp"
2   #include "core/Random.hpp"
3   #include "core/Universe.hpp"
4   #include "ui/Input.hpp"
5   #include <algorithm>
6   #include <iostream>
7   #include <sstream>
8
9   static const size_t TABLE_LIMIT = 32;
10  static const size_t PRINT_LIMIT = 16;
11  static const size_t LINE_WIDTH = 100;
12  static const size_t MATRIX_WIDTH = 112;
13
14  struct OperationInfo{
15      const char* title;
16      const char* notation;
17      const char* formula;
18  };
19
20  static const OperationInfo OPERATIONS[] = {
21      {"объединение", "A ∪ B", "max(kA, kB)"},
22      {"пересечение", "A ∩ B", "min(kA, kB)"},
23      {"разность", "A \\ B", "min(kA, kU - kB)"},
24      {"разность", "B \\ A", "min(kB, kU - kA)"},
25      {"симметрическая разность", "A Δ B", "max(k[A\\B], k[B\\A])"},
26      {"дополнение", "¬A", "kU - kA"},
27      {"дополнение", "¬B", "kU - kB"},
28      {"арифметическая сумма", "A + B", "min(kA + kB, kU)"},
29      {"арифметическая разность", "A - B", "max(kA - kB, 0)"},
30      {"арифметическая разность", "B - A", "max(kB - kA, 0)"},
31      {"арифметическое произведение", "A * B", "min(kA * kB, kU)"},
32      {"арифметическое деление", "A / B", "kA div kB; 0 при kB = 0"},
33      {"арифметическое деление", "B / A", "kB div kA; 0 при kA = 0"},
34  };
35  static const size_t OPERATION_COUNT = sizeof(OPERATIONS) / sizeof(OPERATIONS[0]);
36
37  static const char* COMMANDS = "u - универсум, a/b - задать A/B, t - таблица, o - операции,
    "
38                                "m - матрица, h - справка, q - выход";
39
40  static size_t text_width(const std::string& text){
41      size_t width = 0;
42      for(size_t i = 0; i < text.size(); i++){
43          if((static_cast<unsigned char>(text[i]) & 0xC0) != 0x80) width++;
```

```
    }
    return width;
}

static std::string pad_right(const std::string& text, size_t width){
    size_t current = text_width(text);
    return current >= width ? text : text + std::string(width - current, ' ');
}

static std::string pad_left(const std::string& text, size_t width){
    size_t current = text_width(text);
    return current >= width ? text : std::string(width - current, ' ') + text;
}

static std::string format_multiset(const Multiset& multiset){
    size_t depth = multiset.get_carrier().get_depth();
    size_t limit = PRINT_LIMIT;
    if(depth > 4){
        size_t element_width = depth + 6;
        limit = std::min(PRINT_LIMIT, std::max<size_t>(4, LINE_WIDTH / element_width));
    }
    std::ostringstream out;
    print_multiset(out, multiset, limit);
    return out.str();
}

static std::string compact(const std::string& notation){
    std::string res = notation;
    res.erase(std::remove(res.begin(), res.end(), ' '), res.end());
    return res;
}

Menu::Menu():carrier(std::make_shared<CodeSet>(0)),universe(carrier),a(carrier),b(carrier),
             universe_ready(false),a_ready(false),b_ready(false){}

void Menu::run(){
    while(true){
        std::cout << std::endl << COMMANDS << std::endl;
        char command = read_letter("> ", "uabtomhq", false);
        if(command == 'q') return;
        if(!is_available(command)) continue;
        switch(command){
        case 'u': create_universe(); break;
        case 'a': fill_multiset(a, a_ready, "A"); break;
        case 'b': fill_multiset(b, b_ready, "B"); break;
        case 't': show_table(); break;
        case 'o': show_operations(); break;
        case 'm': show_matrix(); break;
        case 'h': show_help(); break;
        }
    }
}

bool Menu::is_available(char command) const{
    if(command == 'u' || command == 'h') return true;
    if(!universe_ready){
        std::cout << "! команда " << command << " недоступна: универсум ещё не создан
        (команда u)" << std::endl;
        return false;
    }
```

```
103     if((command == 'o' || command == 'm') && (!a_ready || !b_ready)){
104         std::string missing;
105         if(!a_ready) missing = "A (команда a)";
106         if(!b_ready) missing += std::string(missing.empty() ? "" : " и ") + "B (команда
            b)";
107         std::cout << "! команда " << command << " недоступна: не задано " << missing <<
            std::endl;
108         return false;
109     }
110     return true;
111 }
112 
113 void Menu::create_universe(){
114     size_t depth = read_number("n [0.." + std::to_string(MAX_DEPTH) + "]: ", 0,
        MAX_DEPTH);
115     std::shared_ptr<const CodeSet> new_carrier = std::make_shared<CodeSet>(depth);
116     size_t size = new_carrier->size();
117     Multiset new_universe(new_carrier);
118     size_t multiplicity = 0;
119 
120     if(size == 0){
121         std::cout << "при n = 0 в коде Грея нет слов, кратность не нужна" << std::endl;
122     }
123     else{
124         if(!read_number_or_empty("k [1.." + std::to_string(MAX_MULTIPLICITY) + ", Enter -
            случайно]: ",
125                                  1, MAX_MULTIPLICITY, multiplicity)){
126             multiplicity = 1 + random_below(MAX_MULTIPLICITY);
127             std::cout << "k = " << multiplicity << std::endl;
128         }
129         new_universe = uniform_universe(new_carrier, multiplicity);
130     }
131 
132     bool had_multisets = a_ready || b_ready;
133     carrier = new_carrier;
134     universe = new_universe;
135     a = Multiset(carrier);
136     b = Multiset(carrier);
137     universe_ready = true;
138     a_ready = false;
139     b_ready = false;
140 
141     if(size > 0){
142         size_t shown = std::min(size, PRINT_LIMIT);
143         size_t per_line = std::max<size_t>(1, LINE_WIDTH / (depth + 1));
144         std::cout << "код Грея, 2^" << depth << " = " << size << ":";
145         for(size_t i = 0; i < shown; i++){
146             if(shown > per_line && i % per_line == 0) std::cout << std::endl;
147             else if(i > 0 || shown <= per_line) std::cout << " ";
148             std::cout << carrier->get_code(i);
149         }
150         std::cout << std::endl;
151         if(shown < size) std::cout << "... и ещё " << size - shown << std::endl;
152     }
153     std::cout << "U = " << format_multiset(universe) << ", |U| = " <<
        universe.cardinality() << std::endl;
154     if(had_multisets) std::cout << "A и B сброшены" << std::endl;
155 }
156 
157 void Menu::fill_multiset(Multiset& multiset, bool& ready, const std::string& name){
158     size_t high = universe.cardinality();
```

```
159     size_t cardinality = 0;
160     if(!read_number_or_empty("|" + name + "| [0.." + std::to_string(high) + ", Enter -
        случайно]: ",
161                              0, high, cardinality)){
162         cardinality = random_below(high + 1);
163         std::cout << "|" << name << "| = " << cardinality << std::endl;
164     }
165     char method = read_letter("r - случайно, m - вручную, Enter - отмена: ", "rm", true);
166     if(method == '\0'){
167         std::cout << "отменено" << std::endl;
168         return;
169     }
170 
171     Multiset filled(carrier);
172     if(method == 'm'){
173         if(!fill_manual(filled, cardinality, name)){
174             std::cout << "отменено" << std::endl;
175             return;
176         }
177     }
178     else{
179         filled = random_submultiset(universe, cardinality);
180     }
181 
182     multiset = filled;
183     ready = true;
184     std::cout << name << " = " << format_multiset(multiset) << ", |" << name << "| = "
185               << multiset.cardinality() << std::endl;
186 }
187 
188 bool Menu::fill_manual(Multiset& target, size_t cardinality, const std::string& name)
    const{
189     if(cardinality == 0) return true;
190     std::cout << "пары код:кратность через пробел (01:2 11), каждого кода не больше "
191               << universe.get_count(0) << ", Enter - отмена" << std::endl;
192     while(target.cardinality() < cardinality){
193         size_t left = cardinality - target.cardinality();
194         std::string line = read_text(name + " = " + format_multiset(target) + ", осталось
            " + std::to_string(left) + ": ");
195         if(line.empty()) return false;
196         std::vector<size_t> indices;
197         std::vector<size_t> counts;
198         if(parse_manual_line(line, target, left, indices, counts)){
199             for(size_t i = 0; i < indices.size(); i++){
200                 target.set_count(indices[i], target.get_count(indices[i]) + counts[i]);
201             }
202         }
203     }
204     return true;
205 }
206 
207 bool Menu::parse_manual_line(const std::string& line, const Multiset& target, size_t left,
208                              std::vector<size_t>& indices, std::vector<size_t>& counts)
                             const{
209     size_t depth = carrier->get_depth();
210     size_t total = 0;
211     std::istringstream words(line);
212     std::string word;
213     while(words >> word){
214         size_t colon = word.find(':');
215         std::string code = word.substr(0, colon);
```

```
216         size_t count = 1;
217         if(!is_code(code, depth)){
218             std::cout << "! «" << word << "»: код должен состоять из " << depth << "
                символов 0 и 1" << std::endl;
219             return false;
220         }
221         if(colon != std::string::npos && (!parse_number(word.substr(colon + 1), count) ||
            count == 0)){
222             std::cout << "! «" << word << "»: после двоеточия нужна кратность - целое
                число от 1" << std::endl;
223             return false;
224         }
225         size_t index = 0;
226         carrier->find(code, index);
227         size_t taken = target.get_count(index);
228         for(size_t i = 0; i < indices.size(); i++){
229             if(indices[i] == index) taken += counts[i];
230         }
231         size_t free = universe.get_count(index) - std::min(taken,
            universe.get_count(index));
232         if(count > free){
233             std::cout << "! «" << word << "»: у " << code << " свободно " << free << " (kU
                = "
234                       << universe.get_count(index) << ", занято " << taken << ")" <<
                       std::endl;
235             return false;
236         }
237         indices.push_back(index);
238         counts.push_back(count);
239         total += count;
240     }
241     if(total > left){
242         std::cout << "! в строке " << total << " ед., а осталось распределить " << left <<
            std::endl;
243         return false;
244     }
245     return true;
246 }
247
248 std::vector<Multiset> Menu::compute_operations() const{
249     std::vector<Multiset> res;
250     res.push_back(a.unite(b));
251     res.push_back(a.intersect(b));
252     res.push_back(a.difference(b, universe));
253     res.push_back(b.difference(a, universe));
254     res.push_back(a.symmetric_difference(b, universe));
255     res.push_back(a.complement(universe));
256     res.push_back(b.complement(universe));
257     res.push_back(a.arithmetic_sum(b, universe));
258     res.push_back(a.arithmetic_difference(b));
259     res.push_back(b.arithmetic_difference(a));
260     res.push_back(a.arithmetic_product(b, universe));
261     res.push_back(a.arithmetic_division(b));
262     res.push_back(b.arithmetic_division(a));
263     return res;
264 }
265
266 void Menu::show_help() const{
267     std::cout << "запись {000×2, 011×1}: код Грея × кратность, {} - пустое; kU, kA, kB -
        кратности в U, A, B" << std::endl;
268     for(size_t i = 0; i < OPERATION_COUNT; i++){
```

```
269         std::cout << pad_right(OPERATIONS[i].notation, 7) <<
            pad_right(OPERATIONS[i].formula, 26)
270                   << OPERATIONS[i].title << std::endl;
271     }
272     std::cout << "0 <= n <= " << MAX_DEPTH << ", 1 <= k <= " << MAX_MULTIPLICITY << ", 0
        <= |A|, |B| <= |U|" << std::endl;
273 }
274
275 void Menu::show_table() const{
276     size_t code_width = std::max<size_t>(carrier->get_depth(), 3);
277     size_t number_width = std::max<size_t>(std::to_string(universe.cardinality()).size(),
        3) + 1;
278     std::cout << pad_right("код", code_width) << pad_left("kU", number_width)
279               << pad_left("kA", number_width) << pad_left("kB", number_width) <<
                  std::endl;
280     size_t rows = std::min(universe.size(), TABLE_LIMIT);
281     for(size_t i = 0; i < rows; i++){
282         std::cout << pad_right(carrier->get_code(i), code_width)
283                   << pad_left(std::to_string(universe.get_count(i)), number_width)
284                   << pad_left(a_ready ? std::to_string(a.get_count(i)) : "-",
                      number_width)
285                   << pad_left(b_ready ? std::to_string(b.get_count(i)) : "-",
                      number_width) << std::endl;
286     }
287     if(rows < universe.size()){
288         std::cout << "... показаны " << rows << " из " << universe.size() << std::endl;
289     }
290     std::cout << pad_right("Σ", code_width)
291               << pad_left(std::to_string(universe.cardinality()), number_width)
292               << pad_left(a_ready ? std::to_string(a.cardinality()) : "-", number_width)
293               << pad_left(b_ready ? std::to_string(b.cardinality()) : "-", number_width)
                  << std::endl;
294 }
295
296 void Menu::show_operations() const{
297     std::vector<Multiset> results = compute_operations();
298     for(size_t i = 0; i < OPERATION_COUNT; i++){
299         std::cout << pad_right(OPERATIONS[i].notation, 6) << "= " <<
            format_multiset(results[i])
300                   << ", |" << OPERATIONS[i].notation << "| = " << results[i].cardinality()
                      << std::endl;
301     }
302
303     size_t u = universe.cardinality();
304     size_t na = a.cardinality();
305     size_t nb = b.cardinality();
306     size_t complement = results[5].cardinality();
307     size_t unite = results[0].cardinality();
308     size_t intersect = results[1].cardinality();
309     std::cout << "проверка: |¬A| = |U| - |A|: " << complement << " = " << u << " - " << na
310               << (complement == u - na ? " верно" : " НЕВЕРНО") << std::endl;
311     std::cout << "проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: " << unite << " + " <<
        intersect << " = " << na << " + " << nb
312               << (unite + intersect == na + nb ? " верно" : " НЕВЕРНО") << std::endl;
313 }
314
315 void Menu::show_matrix() const{
316     std::vector<Multiset> results = compute_operations();
317     std::vector<std::string> titles = {"U", "A", "B"};
318     std::vector<const Multiset*> rows = {&universe, &a, &b};
319     for(size_t i = 0; i < OPERATION_COUNT; i++){
```

```
        titles.push_back(compact(OPERATIONS[i].notation));
        rows.push_back(&results[i]);
    }

    const size_t label_width = 4;
    const size_t total_width = 9;
    size_t column_width = std::max<size_t>(carrier->get_depth(), 3) + 2;
    size_t columns = std::min(universe.size(), (MATRIX_WIDTH - label_width - total_width)
    / column_width);

    std::cout << pad_right("", label_width);
    for(size_t j = 0; j < columns; j++) std::cout << pad_left(carrier->get_code(j),
    column_width);
    std::cout << pad_left("|·|", total_width) << std::endl;
    for(size_t i = 0; i < rows.size(); i++){
        std::cout << pad_right(titles[i], label_width);
        for(size_t j = 0; j < columns; j++){
            std::cout << pad_left(std::to_string(rows[i]->get_count(j)), column_width);
        }
        std::cout << pad_left(std::to_string(rows[i]->cardinality()), total_width) <<
        std::endl;
    }
    if(columns < universe.size()){
        std::cout << "... показаны " << columns << " из " << universe.size() << " кодов,
        мощности по всем" << std::endl;
    }
}
```

Листинг 9 — `src/core/GrayCode.hpp`

```
#pragma once
#include <cstddef>
#include <stdexcept>
#include <string>
#include <vector>

const size_t MAX_DEPTH = 19;

const size_t MAX_TYPE_DEPTH = sizeof(size_t) * 8 - 1;

class GrayCodeError : public std::runtime_error{
public:
    explicit GrayCodeError(const std::string& message):std::runtime_error(message){}
};

class InvalidDepthError : public GrayCodeError{
public:
    InvalidDepthError():GrayCodeError("2^n не помещается в size_t"){}
};

class CodeIndexError : public GrayCodeError{
public:
    CodeIndexError():GrayCodeError("номер кода вне последовательности"){}
};

std::vector<size_t> generate_gray_code(size_t depth);

std::string code_to_string(size_t code, size_t depth);
```

```
29
30 class CodeSet{
31     size_t depth;
32     std::vector<size_t> codes;
33     std::vector<size_t> positions;
34
35 public:
36     explicit CodeSet(size_t depth);
37
38     size_t get_depth() const;
39     size_t size() const;
40     std::string get_code(size_t i) const;
41     bool find(const std::string& code, size_t& index) const;
42 };
```

Листинг 10 — src/core/GrayCode.cpp

```
1  #include "core/GrayCode.hpp"
2
3  static size_t changed_bit(size_t i){
4      size_t q = 0;
5      size_t j = i;
6      while(j % 2 == 0){
7          j /= 2;
8          q++;
9      }
10     return q;
11 }
12
13 std::vector<size_t> generate_gray_code(size_t depth){
14     if(depth > MAX_TYPE_DEPTH){
15         throw InvalidDepthError();
16     }
17     std::vector<size_t> codes;
18     if(depth == 0){
19         return codes;
20     }
21     size_t count = size_t(1) << depth;
22     codes.reserve(count);
23     size_t code = 0;
24     codes.push_back(code);
25     for(size_t i = 1; i < count; i++){
26         size_t p = changed_bit(i);
27         code ^= size_t(1) << p;
28         codes.push_back(code);
29     }
30     return codes;
31 }
32
33 std::string code_to_string(size_t code, size_t depth){
34     std::string res;
35     for(size_t bit = depth; bit-- > 0;){
36         res += ((code >> bit) & 1) ? '1' : '0';
37     }
38     return res;
39 }
40
41 CodeSet::CodeSet(size_t
   depth):depth(depth),codes(generate_gray_code(depth)),positions(codes.size()){
```

```
    for(size_t i = 0; i < codes.size(); i++){
        positions[codes[i]] = i;
    }
}

size_t CodeSet::get_depth() const{
    return depth;
}

size_t CodeSet::size() const{
    return codes.size();
}

std::string CodeSet::get_code(size_t i) const{
    if(i >= codes.size()){
        throw CodeIndexError();
    }
    return code_to_string(codes[i], depth);
}

bool CodeSet::find(const std::string& code, size_t& index) const{
    if(codes.empty() || code.size() != depth){
        return false;
    }
    size_t value = 0;
    for(size_t i = 0; i < code.size(); i++){
        if(code[i] != '0' && code[i] != '1') return false;
        value = value * 2 + (code[i] - '0');
    }
    index = positions[value];
    return true;
}
```

Листинг 11 — `src/core/Multiset.hpp`

```
#pragma once
#include "core/GrayCode.hpp"
#include <cstddef>
#include <memory>
#include <ostream>
#include <stdexcept>
#include <string>
#include <vector>

class MultisetError : public std::runtime_error{
public:
    explicit MultisetError(const std::string& message):std::runtime_error(message){}
};

class MultisetIndexError : public MultisetError{
public:
    MultisetIndexError():MultisetError("номер элемента вне носителя"){}
};

class CarrierMismatchError : public MultisetError{
public:
    CarrierMismatchError():MultisetError("мультимножества построены над разными
    носителями"){}
```

```
};

class NotSubmultisetError : public MultisetError{
public:
    NotSubmultisetError():MultisetError("кратность элемента больше, чем в универсуме"){}
};

class CardinalityOverflowError : public MultisetError{
public:
    CardinalityOverflowError():MultisetError("мощность больше мощности ограничивающего
    мультимножества"){}
};

class Multiset{
    std::shared_ptr<const CodeSet> carrier;
    std::vector<size_t> counts;

    void check_same_carrier(const Multiset& other) const;
    void check_within(const Multiset& universe) const;

public:
    explicit Multiset(std::shared_ptr<const CodeSet> carrier);

    const CodeSet& get_carrier() const;
    size_t size() const;
    size_t get_count(size_t i) const;
    void set_count(size_t i, size_t count);
    size_t cardinality() const;
    bool is_submultiset_of(const Multiset& other) const;

    Multiset unite(const Multiset& b) const;
    Multiset intersect(const Multiset& b) const;
    Multiset complement(const Multiset& universe) const;
    Multiset difference(const Multiset& b, const Multiset& universe) const;
    Multiset symmetric_difference(const Multiset& b, const Multiset& universe) const;

    Multiset arithmetic_sum(const Multiset& b, const Multiset& universe) const;
    Multiset arithmetic_difference(const Multiset& b) const;
    Multiset arithmetic_product(const Multiset& b, const Multiset& universe) const;
    Multiset arithmetic_division(const Multiset& b) const;

    friend Multiset random_submultiset(const Multiset& bound, size_t cardinality);
};

Multiset random_submultiset(const Multiset& bound, size_t cardinality);

void print_multiset(std::ostream& out, const Multiset& multiset, size_t limit);
std::ostream& operator<<(std::ostream& out, const Multiset& multiset);
```

Листинг 12 — `src/core/Multiset.cpp`

```
#include "core/Multiset.hpp"
#include "core/Random.hpp"
#include <algorithm>

Multiset::Multiset(std::shared_ptr<const CodeSet>
carrier):carrier(carrier),counts(carrier->size(), 0){}

```

```
7  void Multiset::check_same_carrier(const Multiset& other) const{
8      if(carrier != other.carrier){
9          throw CarrierMismatchError();
10     }
11 }
12
13 void Multiset::check_within(const Multiset& universe) const{
14     if(!is_submultiset_of(universe)){
15         throw NotSubmultisetError();
16     }
17 }
18
19 const CodeSet& Multiset::get_carrier() const{
20     return *carrier;
21 }
22
23 size_t Multiset::size() const{
24     return counts.size();
25 }
26
27 size_t Multiset::get_count(size_t i) const{
28     if(i >= counts.size()){
29         throw MultisetIndexError();
30     }
31     return counts[i];
32 }
33
34 void Multiset::set_count(size_t i, size_t count){
35     if(i >= counts.size()){
36         throw MultisetIndexError();
37     }
38     counts[i] = count;
39 }
40
41 size_t Multiset::cardinality() const{
42     size_t res = 0;
43     for(size_t i = 0; i < counts.size(); i++){
44         res += counts[i];
45     }
46     return res;
47 }
48
49 bool Multiset::is_submultiset_of(const Multiset& other) const{
50     check_same_carrier(other);
51     for(size_t i = 0; i < counts.size(); i++){
52         if(counts[i] > other.counts[i]) return false;
53     }
54     return true;
55 }
56
57 Multiset Multiset::unite(const Multiset& b) const{
58     check_same_carrier(b);
59     Multiset res(carrier);
60     for(size_t i = 0; i < counts.size(); i++){
61         res.counts[i] = std::max(counts[i], b.counts[i]);
62     }
63     return res;
64 }
65
66 Multiset Multiset::intersect(const Multiset& b) const{
67     check_same_carrier(b);
68     Multiset res(carrier);
```

```
69      for(size_t i = 0; i < counts.size(); i++){
70          res.counts[i] = std::min(counts[i], b.counts[i]);
71      }
72      return res;
73  }
74
75  Multiset Multiset::complement(const Multiset& universe) const{
76      check_within(universe);
77      Multiset res(carrier);
78      for(size_t i = 0; i < counts.size(); i++){
79          res.counts[i] = universe.counts[i] - counts[i];
80      }
81      return res;
82  }
83
84  Multiset Multiset::difference(const Multiset& b, const Multiset& universe) const{
85      return intersect(b.complement(universe));
86  }
87
88  Multiset Multiset::symmetric_difference(const Multiset& b, const Multiset& universe)
    const{
89      return difference(b, universe).unite(b.difference(*this, universe));
90  }
91
92  Multiset Multiset::arithmetic_sum(const Multiset& b, const Multiset& universe) const{
93      check_within(universe);
94      b.check_within(universe);
95      Multiset res(carrier);
96      for(size_t i = 0; i < counts.size(); i++){
97          res.counts[i] = std::min(counts[i] + b.counts[i], universe.counts[i]);
98      }
99      return res;
100 }
101
102 Multiset Multiset::arithmetic_difference(const Multiset& b) const{
103     check_same_carrier(b);
104     Multiset res(carrier);
105     for(size_t i = 0; i < counts.size(); i++){
106         res.counts[i] = (counts[i] > b.counts[i]) ? counts[i] - b.counts[i] : 0;
107     }
108     return res;
109 }
110
111 Multiset Multiset::arithmetic_product(const Multiset& b, const Multiset& universe) const{
112     check_within(universe);
113     b.check_within(universe);
114     Multiset res(carrier);
115     for(size_t i = 0; i < counts.size(); i++){
116         res.counts[i] = std::min(counts[i] * b.counts[i], universe.counts[i]);
117     }
118     return res;
119 }
120
121 Multiset Multiset::arithmetic_division(const Multiset& b) const{
122     check_same_carrier(b);
123     Multiset res(carrier);
124     for(size_t i = 0; i < counts.size(); i++){
125         res.counts[i] = (b.counts[i] != 0) ? counts[i] / b.counts[i] : 0;
126     }
127     return res;
128 }
129
```

```
Multiset random_submultiset(const Multiset& bound, size_t cardinality){
    if(cardinality > bound.cardinality()){
        throw CardinalityOverflowError();
    }
    Multiset res(bound.carrier);
    std::vector<size_t> free;
    free.reserve(bound.counts.size());
    for(size_t i = 0; i < bound.counts.size(); i++){
        if(bound.counts[i] > 0) free.push_back(i);
    }
    for(size_t step = 0; step < cardinality; step++){
        size_t j = random_below(free.size());
        size_t i = free[j];
        res.counts[i]++;
        if(res.counts[i] == bound.counts[i]){
            free[j] = free.back();
            free.pop_back();
        }
    }
    return res;
}

void print_multiset(std::ostream& out, const Multiset& multiset, size_t limit){
    const CodeSet& carrier = multiset.get_carrier();
    size_t shown = 0;
    size_t hidden = 0;
    out << "{";
    for(size_t i = 0; i < multiset.size(); i++){
        size_t count = multiset.get_count(i);
        if(count == 0) continue;
        if(shown == limit){
            hidden++;
            continue;
        }
        if(shown > 0) out << ", ";
        out << carrier.get_code(i) << "×" << count;
        shown++;
    }
    if(hidden > 0) out << ", ... ещё " << hidden << " эл.";
    out << "}";
}

std::ostream& operator<<(std::ostream& out, const Multiset& multiset){
    print_multiset(out, multiset, multiset.size());
    return out;
}
```

Листинг 13 — `src/core/Random.hpp`

```
#pragma once
#include <cstddef>

unsigned make_seed();

void seed_random(unsigned seed);

size_t random_below(size_t bound);
```

Листинг 14 — src/core/Random.cpp

```
#include "core/Random.hpp"
#include <chrono>
#include <random>

static std::mt19937& generator(){
    static std::mt19937 gen;
    return gen;
}

unsigned make_seed(){
    std::random_device device;
    unsigned time = static_cast<unsigned>(
        std::chrono::high_resolution_clock::now().time_since_epoch().count());
    return device() ^ time;
}

void seed_random(unsigned seed){
    generator().seed(seed);
}

size_t random_below(size_t bound){
    std::uniform_int_distribution<size_t> distribution(0, bound - 1);
    return distribution(generator());
}
```

Листинг 15 — src/core/Universe.hpp

```
#pragma once
#include "core/GrayCode.hpp"
#include "core/Multiset.hpp"
#include <cstddef>
#include <memory>

const size_t MAX_MULTIPLICITY = 100;

class InvalidMultiplicityError : public MultisetError{
public:
    InvalidMultiplicityError():MultisetError("кратность элемента универсума вне диапазона
    от 1 до MAX_MULTIPLICITY"){}
};

Multiset uniform_universe(std::shared_ptr<const CodeSet> carrier, size_t multiplicity);
```

Листинг 16 — src/core/Universe.cpp

```
#include "core/Universe.hpp"

Multiset uniform_universe(std::shared_ptr<const CodeSet> carrier, size_t multiplicity){
    if(multiplicity < 1 || multiplicity > MAX_MULTIPLICITY){
        throw InvalidMultiplicityError();
    }
    Multiset universe(carrier);
    for(size_t i = 0; i < universe.size(); i++){
        universe.set_count(i, multiplicity);
    }
    return universe;
}
```

Листинг 17 — tools/bench.cpp

```
#include "core/GrayCode.hpp"
#include "core/Multiset.hpp"
#include "core/Random.hpp"
#include "core/Universe.hpp"
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <iostream>
#include <memory>

template<class Action>
static double measure(Action action){
    double best = 1e18;
    for(int run = 0; run < 3; run++){
        auto start = std::chrono::steady_clock::now();
        action();
        auto finish = std::chrono::steady_clock::now();
        best = std::min(best, std::chrono::duration<double, std::milli>(finish -
        start).count());
    }
    return best;
}

int main(){
    seed_random(1);
    std::cout << " n   код Грея   универсум   заполнение A   13 операций   наибольшее, мс"
    << std::endl;
    for(size_t depth = 10; depth <= MAX_DEPTH + 2; depth++){
        std::shared_ptr<const CodeSet> carrier;
        double t_code = measure([&]{ carrier = std::make_shared<CodeSet>(depth); });

        Multiset universe(carrier);
        double t_universe = measure([&]{ universe = uniform_universe(carrier,
        MAX_MULTIPLICITY); });

        Multiset a(carrier);
        Multiset b(carrier);
        double t_fill = measure([&]{ a = random_submultiset(universe,
        universe.cardinality()); });
```

```
36          a = random_submultiset(universe, universe.cardinality() / 2);
37          b = random_submultiset(universe, universe.cardinality() / 3);
38
39          size_t checksum = 0;
40          double t_operations = measure([&]{
41              checksum += a.unite(b).cardinality() + a.intersect(b).cardinality()
42                       + a.difference(b, universe).cardinality() + b.difference(a,
                         universe).cardinality()
43                       + a.symmetric_difference(b, universe).cardinality()
44                       + a.complement(universe).cardinality() +
                         b.complement(universe).cardinality()
45                       + a.arithmetic_sum(b, universe).cardinality() +
                         a.arithmetic_difference(b).cardinality()
46                       + b.arithmetic_difference(a).cardinality() + a.arithmetic_product(b,
                         universe).cardinality()
47                       + a.arithmetic_division(b).cardinality() +
                         b.arithmetic_division(a).cardinality();
48          });
49
50          double worst = std::max({t_code, t_universe, t_fill, t_operations});
51          std::cout << std::fixed << std::setprecision(1) << std::setw(2) << depth
52                    << std::setw(11) << t_code << std::setw(12) << t_universe <<
                      std::setw(15) << t_fill
53                    << std::setw(14) << t_operations << std::setw(17) << worst
54                    << (worst < 1000 ? "" : "  > 1 c") << std::endl;
55          if(checksum == 0) std::cout << "";
56      }
57      return 0;
58  }
```